Валентин Юльевич Арьков

Частотный анализ числовых и текстовых данных

Учебное пособие



Валентин Юльевич Арьков

Частотный анализ числовых и текстовых данных

Учебное пособие

Издательские решения
По лицензии Ridero
2023

УДК 00
ББК я92
А89

Шрифты предоставлены компанией «ПараТайп»

Арьков Валентин Юльевич

А89 Частотный анализ числовых и текстовых данных : Учебное пособие / Валентин Юльевич Арьков. — [б. м.] : Издательские решения, 2023. — 114 с.
ISBN 978-5-0059-8841-6

Данная работа посвящена применению современных методов и технологий больших данных в курсе бизнес-аналитике и статистики. Анализ частоты появления различных значений — один из способов первоначальной обработки данных. Чаще всего его относят к описательной статистике или аналитике. Подсчет частот имеет отношение распределению. Поэтому для успешного восприятия материала желательно освежить в памяти основы теории вероятностей и математической статистики.

УДК 00
ББК я92

12+ В соответствии с ФЗ от 29.12.2010 №436-ФЗ

ISBN 978-5-0059-8841-6

© Валентин Юльевич Арьков, 2023

ОГЛАВЛЕНИЕ

Предисловие 5
Введение 6
1. Начало работы 7
1.1. Результаты и защита 9
1.2. Работа и отчет 10
1.3. Создание блокнота 11
2. Анализ числовых данных 13
2.1. Генерируем данные 15
2.2. Гистограмма 19
2.3. Распределение 24
2.4. Полигон 28
2.5. Кумулята 31
2. Анализ текстовых данных 37
2.1. Обработка текстов на естественном языке 39
2.2. Облако слов (тегов) 42
2.3. Маска 46
2.4. Облако фраз 52
2.5. Словарь весов (частот) 55
2.6. Расшифровка видео Youtube 59
2.6.1. Страница видеоролика 60
2.6.2. Субтитры DownSub 62
2.6.3. Студия Youtube 63
2.7. Предварительный анализ текста 66
2.8. Облако слов 72
2.9. Лемматизация 84
2.10. Стоп-слова 90
2.11. Персонализация стоп-слов 94
2.12. Гистограмма 103
2.13. Частотный анализ текста книги 106
Благодарность 110
Ссылки 111
Литература 112
Послесловие 113

ПРЕДИСЛОВИЕ

Данная работа посвящена применению современных методов и технологий больших данных в курсе бизнес-аналитике и статистики. Анализ частоты появления различных значений — один из способов первоначальной обработки данных. Чаще всего его относят к описательной статистике (Descriptive Statistics). В последнее время также говорят об описательной аналитике (Descriptive Analytics). Подсчет частот имеет отношение распределению. Поэтому его восприятия материала желательно обладать пониманием основ теории вероятностей и математической статистики.

В данной работе мы рассмотрим примеры и приемы частотного анализа числовых и текстовых данных средствами **Python**.

К этому материалу следует относиться именно как к примерам. Потому что есть много способов выполнить одно и то же действие. Разные способы и разные алгоритмы могут дать в конечном счете один и тот же результат.

Данное пособие можно скачать совершенно бесплатно через сайт издательства Ридеро. Причем в разных форматах.

ВВЕДЕНИЕ

Частотный анализ — это изучение характеристик распределения случайной величины. Напомним, что распределение описывает вероятность (частоту) появления тех или иных значений статистического признака. Анализ формы и параметров распределения обычно относят к описательной статистике — **Descriptive Statistics**.

Язык Python во многом построен как объектно-ориентированный, поэтому мы имеем дело с объектами. А у объектов есть **методы**, которые соответствуют понятию **функция** и **команда** в традиционном программировании. Для упрощения мы будем говорить «функция», не углубляясь в технические подробности.

Мы будем выполнять работу в облачной среде **Google Colab**, но программу можно также использовать (с небольшими доработками) и в других средах. Основное преимущество **Colab** — это возможность приступить к работе, не занимаясь установкой программного обеспечения. Требования к компьютеру минимальные: наличие браузера и подключения к интернету.

1. Начало работы

1.1. РЕЗУЛЬТАТЫ И ЗАЩИТА

Здесь мы обсудим первые шаги и основные моменты, касающиеся выполнения работы. В данной работе мы проводим обучение в режиме **мастер-класса**, когда студенты повторяют действия специалиста — «мастера своего дела». В данном выражении слово «класс» означает «занятия на выбранную тему». Это выражение позаимствовано из английского. Фактически, это английское выражение **master class**, записанное русскими буквами, а вовсе не перевод.

Программа на **Python** уже готова, её можно просмотреть, скачать и запустить. Мы обсуждаем её по частям, которые называются **ячейками**. Студентам нужно с этой программой ознакомиться и «поиграться», меняя некоторые параметры. В конечном счёте нужно достичь некоторого понимания и способности использовать и модифицировать эту готовую программу.

Поэтому в тексте даются пояснения и несложные задания. Во время защиты студент демонстрирует понимание материала и способность осмысленно использовать готовые команды и конструкции языка.

За счет такого подхода можно быстро войти в курс дела и сделать первые шаги в аналитике данных на Питоне.

Задание. Просмотрите в Википедии статью **Мастер-класс**.

1.2. РАБОТА И ОТЧЕТ

Будем выполнять работу в облачном блокноте **Jupyter Notebook** в среде **Google Colab**.

https://colab.research.google.com/

Для работы понадобится подключение к интернет и учетная запись **Google**. Наш блокнот одновременно является инструментом для анализа данных и отчетом, в котором мы «документируем» все наши действия — шаг за шагом.

Для работы на локальном компьютере можно использовать бесплатную среду **Anaconda** с аналогичными возможностями.

https://www.anaconda.com/

Еще один вариант — это запуск на локальном компьютере контейнера с **Anaconda** и работа с ним через браузер.

1.3. СОЗДАНИЕ БЛОКНОТА

При открытии страницы **Colab** выбираем создание нового блокнота: **New Notebook**. Задаём ему информативное название.

Блокнот состоит из ячеек, которые могут содержать программный код — **Code —** или текст — **Text**. Для создания очередной ячейки нажимаем соответствующую кнопку, см. рис.

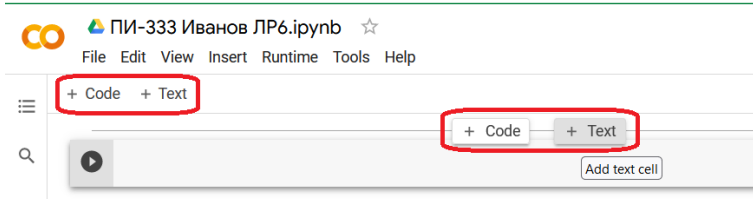


Рис. Создание ячейки

Отчёт по лабораторной работе традиционно начинается с титульного листа. В нашем примере это будет текстовая ячейка, содержащая основные данные о выполненной работе и исполнителе (студенте).

В начале каждого раздела создаём текстовую ячейку с соответствующим названием. Двойным щелчком левой кнопки мыши переходим к редактированию ячейки. Для заголовков первого верхнего уровня используем символ решетки. После ввода текста для «запуска» ячейки на выполнение нажимаем комбинацию клавиш **[Shift + Enter]**. При этом автоматически создаётся новая кодовая ячейка.

Чтобы раскрыть структуру документа, нажимаем кнопку содержание **Table of contents**, см рис.

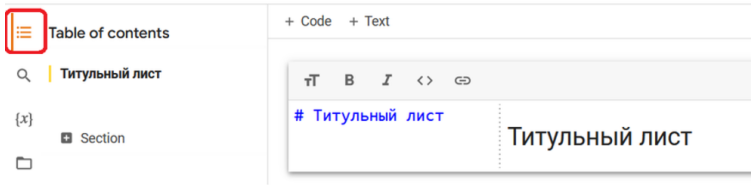


Рис. Заголовок раздела

Для оформления текста можно использовать разметку **Markdown** и **HTML**. При этом в правой половине ячейки открывается предварительный просмотр отформатированного текста, см. рис.

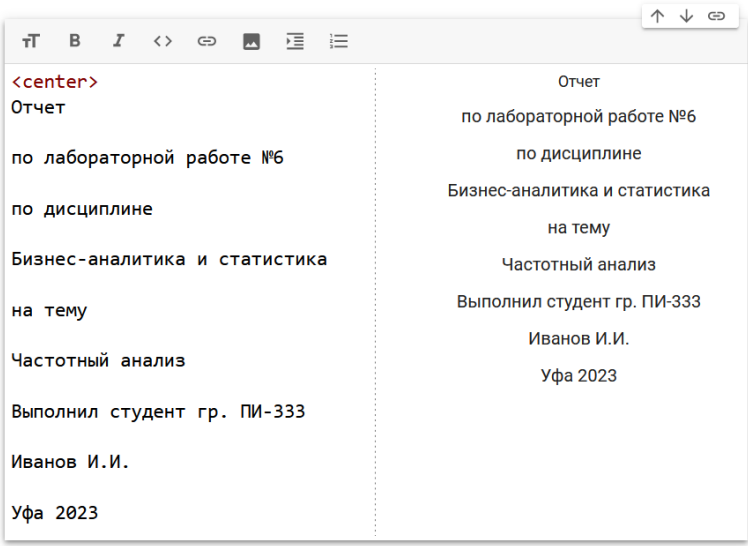


Рис. Титульный лист

2. Анализ числовых данных

2.1. ГЕНЕРИРУЕМ ДАННЫЕ

В этой части работы мы познакомимся с возможностями анализа числовых данных.

Создадим новый раздел и озаглавим его: «Генерируем числовые данные». Обратим внимание, что мы можем перемещаться между разделами документа с помощью оглавления. Мы также можем сворачивать и разворачивать разделы, нажимая кнопку слева от заголовка раздела, см. рис.

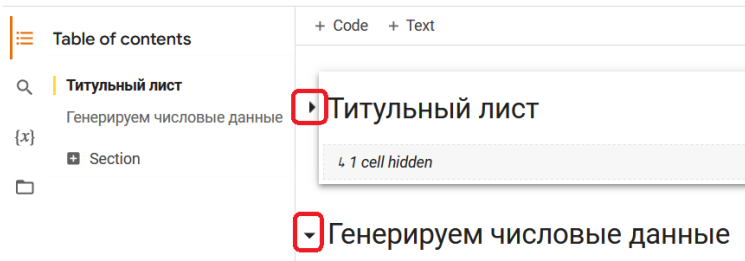


Рис. Управление разделами

Сгенерируем выборку, состоящую из случайных чисел с заданным распределением.

В начале импортируем загружаем библиотеку **numpy** — **Numerical Python Extensions**. Библиотека позволяет работать с числовыми массивами и содержит много полезных математических функций. Здесь же мы назначаем короткий псевдоним **np** для удобства вызова функций, см. рис.

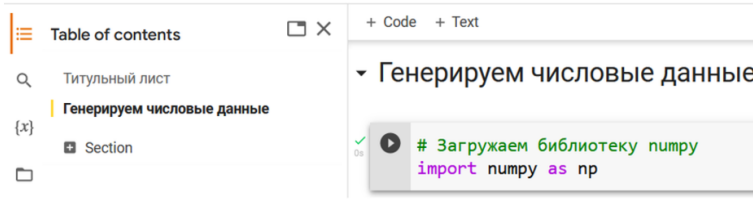


Рис. Загружаем библиотеку

По ходу выполнения работы вставляем комментарии. Это полезно и для себя, и для читателя. Комментарии начинаются с символа решетки **#**. Они сразу позволяют **документировать** ход работы. Так что у нас одновременно появляются и сама программа, и документация — описание к ней.

Зададим начальное состояние генератора случайных чисел. Вызываем функцию **seed** из встроенного модуля **random**. В качестве параметра укажем номер зачетной книжки, см. рис. Задавая одно и то же начальное состояние генератора, мы будет каждый раз получать один и тот же набор данных. На самом деле это псевдослучайные числа, которые получают путём вычислений. Внешне они выглядят как случайные.

```
[1] # Загружаем библиотеку numpy
    import numpy as np

[2] # Номер зачётки - начальное состояние генератора
    np.random.seed(123454)
```

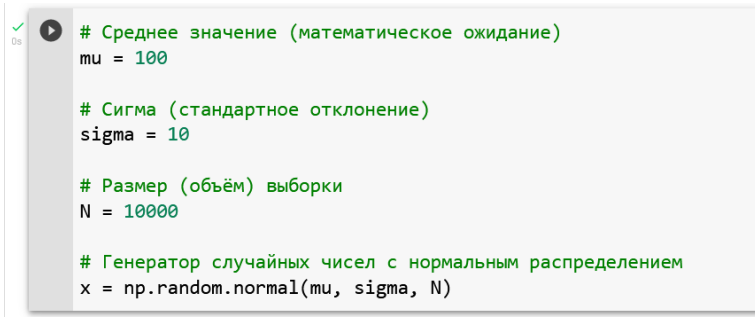
Рис. Инициализация генератора

После выполнения ячейки слева от неё появляется зеленый символ «галочка». Кроме того, в квадратных скобках указано

число. Оно указывает, в каком порядке выполнялись ячейки. В блокноте **Jupyter** можно выполнять ячейки в любом порядке, причем можно запускать их много раз.

Во время работы с блокнотом советуем регулярно нажимать комбинацию клавиш **[Ctrl + S]** для сохранения. Блокнот сохраняется на вашем облачном диске **Google Drive**.

Теперь мы вызываем генератор случайных чисел и передаем ему параметры распределения и объем выборки, см. рис.



```
# Среднее значение (математическое ожидание)
mu = 100

# Сигма (стандартное отклонение)
sigma = 10

# Размер (объём) выборки
N = 10000

# Генератор случайных чисел с нормальным распределением
x = np.random.normal(mu, sigma, N)
```

Рис. Генерируем выборку

Выводим на экран первые 10 элементов массива **x**. Мы не указываем начальный индекс, поэтому используется нулевой элемент. После символа двоеточия указан номер 10, но элемент с номером 10 не выводится. Мы увидим только элементы с номерами от нуля до девяти — это особенность **Python**.

При работе в блокноте нам не обязательно использовать функцию **print**. Достаточно указать имя переменной, см. рис.

```
# Печатаем первые 10 элементов массива
x[:10]
```

```
array([ 99.68533263,  97.65591944,  97.12840918,  97.61284161,
       107.61516112,  98.23726448,  89.07263718, 112.34828826,
       108.95359178, 107.31734988])
```

```
# Используем функцию print
print(x[:10])
```

```
[ 99.68533263  97.65591944  97.12840918  97.61284161 107.61516112
  98.23726448  89.07263718 112.34828826 108.95359178 107.31734988]
```

Рис. Начало массива

2.2. ГИСТОГРАММА

Переходим к построению гистограммы.

Для создания графиков мы будем использовать библиотеку **matplotlib**. Из этой библиотеки мы загружаем модуль **pyplot** и назначаем ему псевдоним **plt**.

Теперь мы можем вызвать функцию построения гистограммы **hist**. В качестве параметра передаём имя нашего массива **x**. На экране появляется изображение гистограммы, причём количество интервалов группировки и их границы выбираются автоматически.

Над графиком выводится дополнительная информация: абсолютное частоты попадания в каждый интервал и границы интервалов группировки, см. рис.

Гистограмма

```
[11] # Загружаем графическую библиотеку
     import matplotlib.pyplot as plt
```

```
[13] # Построим гистограмму с настройками по умолчанию
     plt.hist(x)
```

```
(array([   7.,   83.,  483., 1631., 2952., 2936., 1461.,  386.,   55.,
          6.]), array([ 59.1823327 ,  67.45063739,  75.71894207,  83.98724676,
        92.25555144, 100.52385613, 108.79216081, 117.0604655 ,
       125.32877018, 133.59707487, 141.86537955]), <a list of 10 Patch objects>)
```

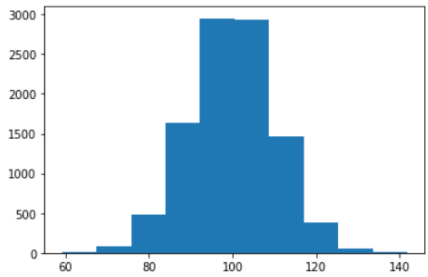


Рис. Гистограмма: настройки по умолчанию

При построении гистограммы мы не указали количество интервалов группировки. По умолчанию используется 10 интервалов. Чтобы задать другое значение, укажем параметр **bins**, см. рис. Нам нужно подобрать «приемлемое» количество интервалов — не слишком мало и не слишком много. Меняем значение **n** и запускаем нашу ячейку на выполнение.

Обратим внимание, что при вызове функций в Python мы можем передавать им параметры в любом порядке и даже можем их пропускать. Чтобы справиться с такими ситуациями, мы явно указываем название параметра, затем ставим знак равенства и даём его значение, например, **bins=n**.

Команда **plt.show ()** подавляет вывод служебной информации на экран. В этом случае мы выводим на экран только сам график.

```
# Подбираем число интервалов группировки
n = 20
# Гистограмма распределения
plt.hist(x, bins=n)
# Выводить только график
plt.show()
```

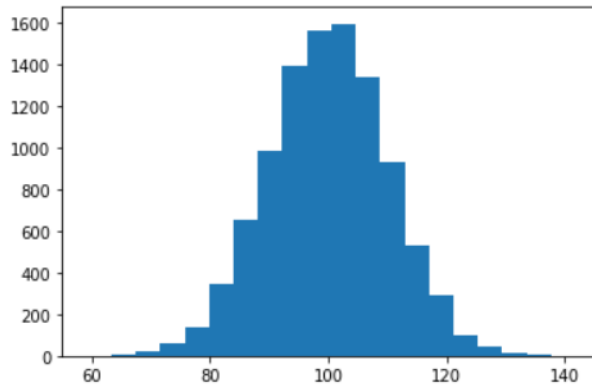


Рис. Настройка числа интервалов

Если задать слишком мало интервалов группировки, получается «грубая» картина с крупными «ступеньками», см. рис., слева. Слишком много интервалов дадут излишние подробности на графике — это просто случайный шум, за которым плохо просматривается форма распределения, см. рис, справа.

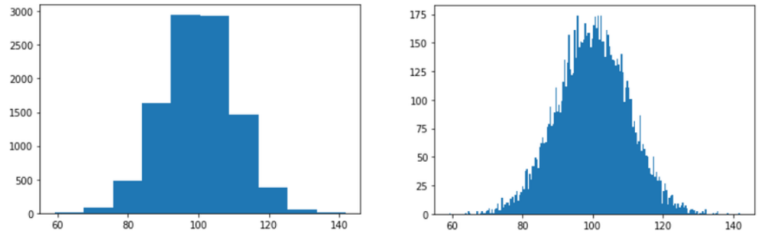


Рис. Подбираем число интервалов

Постепенно увеличиваем количество интервалов, пока не получим компромисс между грубыми ступеньками и случайными отклонениями. График должен получиться достаточно подробным, но не зашумлённым.

Теперь перейдём к оформлению графика, см. рис.

Настроим размеры изображения с помощью функции **plt.figure (figsize= (12, 6))**. Параметр **figsize** получает значения ширины и высоты изображения в дюймах.

Передадим дополнительные параметры функции **plt. hist**:

— **edgecolor='black'** — чёрные границы столбиков;

— **linewidth=2** — толщина линий;

— **facecolor='white'** — заливка столбиков белым цветом;

— **density=True** — пересчитать результаты группировки как плотность вероятности.

Теперь площадь под графиком равна единице, и нашу гистограмму можно будет сравнивать с теоретической кривой нормального распределения.

Наконец, с помощью функции **plt. title** мы выводим заголовок графика.

```
# Размеры графика в дюймах
plt.figure(figsize=(12, 6))
# Цвет и толщина линий, цвет заливки столбиков
plt.hist(x, bins=n, edgecolor='black', linewidth=2, facecolor='white', density=True)
# Заголовок графика
plt.title("Гистограмма нормального распределения")
# Не выводить результаты группировки данных на экран
plt.show()
```

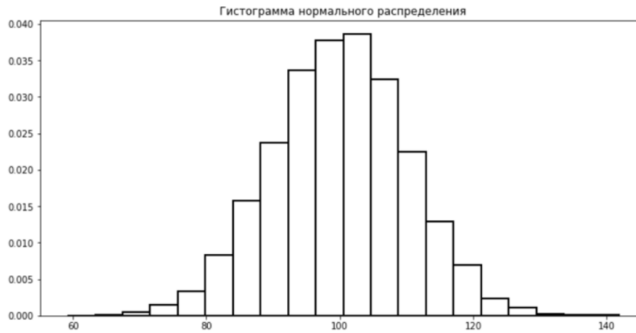


Рис. Оформление графика

2.3. РАСПРЕДЕЛЕНИЕ

Рассчитаем теоретические значения плотности вероятности нормального распределения, см. рис.

Загружаем функцию нормального распределения **norm** из модуля **stat** библиотеки **scipy**.

Далее формируем одномерный массив (вектор) из 100 чисел в диапазоне от 60 до 140. В этих пределах расположена наша гистограмма. Для этого используем функцию **linspace** из библиотеки **numpy**.

Далее вычисляем значения плотности вероятности для нормального распределения с помощью функции **norm. pdf**. Название **pdf** в данном случае расшифровывается как **probability density function** — функция плотности вероятности. Задаем для этой функции те же параметры, что для генератора наших исходных данных:

— **loc=mu** — среднее значение (математическое ожидание);

— **scale=sigma** — стандартное отклонение («сигма»).

Здесь параметр **loc** означает **location**, то «расположение» среднего значения, то есть математического ожидания, которое обычно обозначается греческой буквой «мю».

Параметр **scale** определяет «размеры», или «размах» нашей кривой. Это среднее квадратическое отклонение, или С. К. О., или просто «сигма».

Строим график функции плотности с помощью функции **plt.plot**. Эта функция наносит на график точки и соединяет их прямыми отрезками. Поскольку точек и отрезков много, они сливаются и внешне выглядят как гладкая кривая.

Рассматриваем график и убеждаемся в правильности расчетов — обращаем внимание на среднее и разброс вокруг среднего (примерно три сигмы). В нашем примере это 100 плюс-минус 30, то есть от 70 до 130.

```
# Загружаем нормальное распределение
from scipy.stats import norm

# Массив заданного количества "иксов" для графика
xx = np.linspace(60, 140, 100)

# График плотности вероятности нормального распределения
plt.plot(xx, norm.pdf(xx, loc=mu, scale=sigma))
plt.show()
```

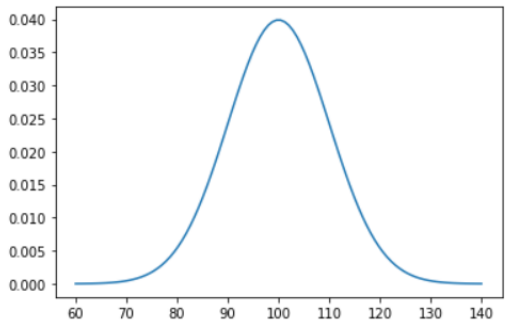


Рис. Плотность вероятности нормального распределения

Завершим оформление гистограммы — добавим теоретическую кривую и немного украсим изображение, см. рис.

При вызове функций **plt. hist** и **plt.plot** мы задаём параметр **label**. Это будет «метка» наших линий. Она нужна для оформления легенды, то есть вывода на график условных обозначений. Легенду мы выводим с помощью функции **plt. legend ()** без параметров.

Для теоретического графика мы выбираем цвет с помощью параметра **color**. Линии разного цвета лучше воспринимаются, и эти цвета будут указаны в легенде.

Далее мы выводим заголовки по осям «икс» и «игрек»: **plt. xlabel** и **plt. ylabel**.

Наконец, на свободном месте графика выводим текст с помощью функции plt. text. Параметры этой функции следующие:

— **x=130, y=0.033** — координаты левого нижнего угла текстового блока;

— **s** — строка текста для вывода на график (естественно, от английского слова **string** — строка).

Чтобы разобраться с этой строкой, придётся сосредоточиться. Итак, вначале указана буква f, что указывается на «форматированную строку», которая дана в кавычках. Далее мы выводим греческую букву «мю», для чего используем средства формул текстового редактора **LaTeX**: **μ**. Формулы здесь вводятся в обрамлении символа «доллара». Затем мы выводим числовое значение переменной **mu**. Для этого используем фигурные скобки: **{mu}**. Затем используем символ перехода на новую строку **\n** — как и в стандартном формате вывода языка Си. Аналогично организуем вывод значения сигмы.

```
# Гистограмма и плотность вероятности нормальноо распределения
plt.figure(figsize=(12, 6))
plt.hist(x, bins=n, edgecolor='black', linewidth=2,
         facecolor='white', density=True, label='Гистограмма')
plt.plot(xx, norm.pdf(xx, loc=mu, scale=sigma),
         color="green", linewidth=2, label="Нормальное распределение")
plt.title("Нормальное распределение")
plt.legend()
plt.xlabel("Значение случайной величины")
plt.ylabel("Плотность вероятности")
plt.text(x=130, y=0.033, s=f"$\mu$ = {mu}\n$\sigma$ = {sigma}")
plt.show()
```

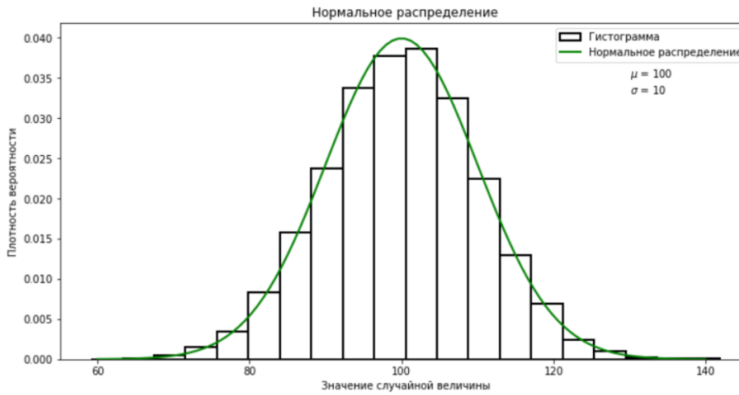


Рис. Оформление гистограммы

2.4. ПОЛИГОН

Вместо гистограммы мы можем построить более гладкий график, который традиционно называется «полигон», то есть «ломаная линия». Дополнительно мы используем сглаживание, которое работает аналогично скользящей средней в динамике. Только здесь сглаживание работает по оси «иксов» вместо оси времени, см. рис.

Загружаем библиотеку **seaborn** и назначаем ей короткий псевдоним **sns**:

import seaborn as sns

Этот инструмент тесно интегрирован с двумя базовыми библиотеками Python: **matplotlib** и **pandas**.

Затем мы вызываем средство для построения графика распределения **sns. displot** — **distribution plot**. Здесь мы выбираем тип графика: **kde** — **Kernel Density Estimate**. Это оценивание формы распределения с использованием «ядерного» сглаживания графика. В данном случае «ядро» (kernel) — это набор весовых коэффициентов для взвешенной скользящей средней. Напомним, что идею простой и взвешенной скользящей средней мы обсуждаем в нашем курсе в разделе «Динамика».

В результате на экран выводится сглаженный график, построенный по нашим исходным данным.

```
import seaborn as sns
```

```
sns.displot(x, kind="kde")
plt.show()
```

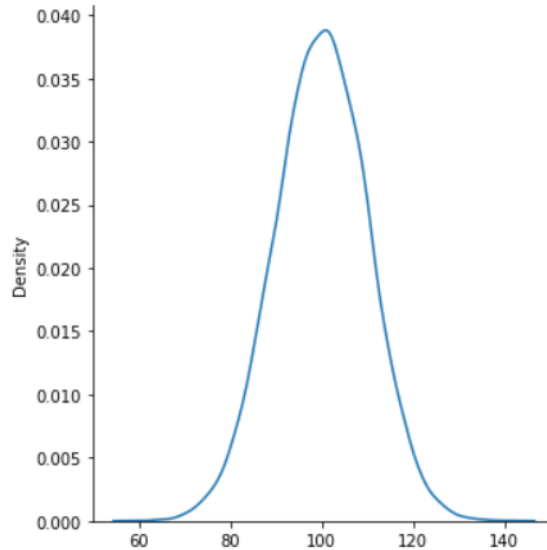


Рис. Сглаженный график плотности

Мы можем управлять степенью сглаживания графика. Для этого имеется параметр **bw_adjust**, см. рис. Название его расшифровывается как **bandwidth** — «полоса пропускания фильтра». Значение по умолчанию: **bw_adjust=1**. Попробуем его уменьшить до 0,1 и увидим появление случайных отклонений.

Подробное описание функции:

https://seaborn.pydata.org/generated/seaborn.kdeplot.html[1]

```
sns.displot(x, kind="kde", bw_adjust=0.1)
plt.show()
```

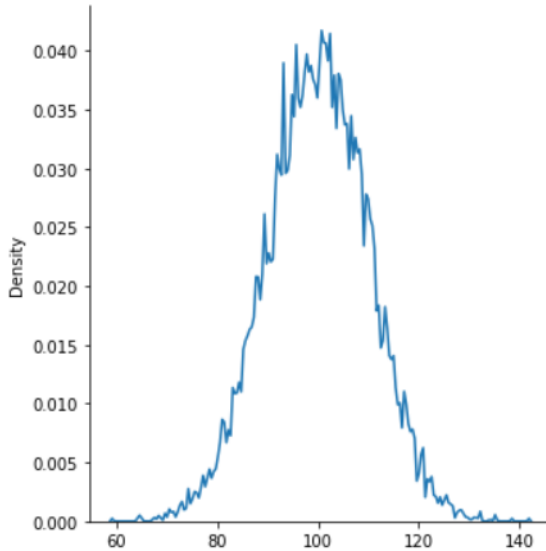


Рис. Управление сглаживанием

[1] https://ridero.ru/link/rsAhBEeu7yxUU04ZVbQ1Z

2.5. КУМУЛЯТА

Переходим ко второму графику для формы распределения. Это график накопленных частот — кумулята.

Используем функцию **plt. hist**. Дополнительно используем следующие параметры:

— **density=True** — пересчет масштаба для приведения графика в соответствие с теорией;

— **cumulative=True** — вычисление накопленных (кумулятивных) частот.

Получаем ступенчатый график, что не вполне соответствует теории вероятностей. На самом деле, кумулята — это интеграл от гистограммы, поэтому нас интересует ломаная линия, а не столбики.

Если в текущей кодовой ячейке не вводить команду **plt.show ()**, на экран выводится служебная информация, см. рис. Здесь первый массив — это значения интегральных вероятностей, второй массив — границы интервалов группировки данных. Эти массивы мы сможем использовать, чтобы построить свой, более грамотный график.

```
plt.hist(x, bins=n, density=True, cumulative=True)
```

```
(array([7.000e-04, 9.000e-03, 5.730e-02, 2.204e-01, 5.156e-01, 8.092e-01,
        9.553e-01, 9.939e-01, 9.994e-01, 1.000e+00]),
 array([ 59.1823327 ,  67.45063739,  75.71894207,  83.98724676,
         92.25555144, 100.52385613, 108.79216081, 117.0604655 ,
        125.32877018, 133.59707487, 141.86537955]),
 <BarContainer object of 10 artists>)
```

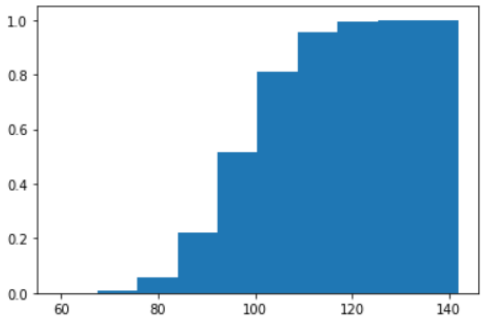


Рис. Столбиковая диаграмма накопленных частот

При вызове функции **plt. hist** мы можем собрать описанные выше служебные данные графика. Для этого мы укажем три переменных в круглых скобках как выходное (возвращаемое) значение, см. рис. Круглые скобки позволяют работать с несколькими переменными как с одним объектом. В Python это называется «кортеж» — **tuple**. После присваивания значений мы сможем использовать эти переменные по отдельности.

Возвращаясь к предыдущему рисунку, обратим внимание, что служебные данные содержат три объекта:

— список накопленных вероятностей;

— список границ интервалов;

— ссылки на столбики графика (**Bar**).

Нам понадобятся первые два объекта.

```
(C_cum, x_cum, p_cum) = plt.hist(x, bins=n,
                                 density=True,
                                 cumulative=True)
```

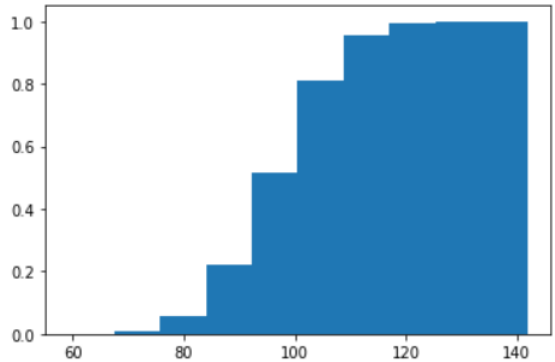


Рис. Сбор исходных данных для графика

Теперь можно заняться окончательным оформлением графика кумуляты (накопленных частот, интегральных вероятностей), см. рис.

```
# Кумулята и функция нормального распределения
plt.figure(figsize=(12, 6))
plt.plot(x_cum[1:], C_cum, color='black', linewidth=2, label='Кумулята', marker='o')
C = norm.cdf(xx, loc=mu, scale=sigma)
plt.plot(xx, C, color="green", linewidth=2, label="Нормальное распределение")
plt.title("Нормальное распределение")
plt.legend()
plt.xlabel("Значение случайной величины")
plt.ylabel("Вероятность (накопленные частоты)")
plt.text(x=63, y=0.8, s=f"$\mu$ = {mu}\n$\sigma$ = {sigma}")
plt.show()
```

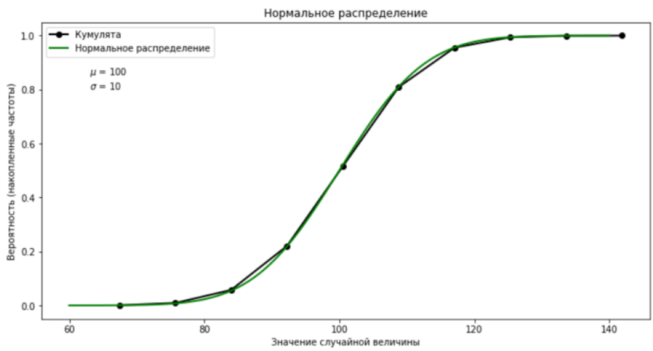


Рис. Оформление графика кумуляты

Первая команды **plt.figure** с параметром **figsize** задаёт размеры графика.

Далее мы строим график кумуляты в виде ломаной линии командой **plt.plot**. В качестве координат по оси «иксов» мы используем список границ интервалов, начиная со второго по счёту элемента: **x_cum [1:]**. Напомним, что в Python нумерация элементов начинается с нуля. Мы выбираем из этого списка только правые границы интервалов. В лекциях и во время практических занятий мы уже подробно обсуждали, почему накопленные частоты нужно привязывать к правым границам интервалов группировки. Смысл в том, что мы получаем количество элементов в интервале только к концу интервала.

В качестве координат по оси «игреков» мы берём значения накопленных частот (интегральных вероятностей) **C_cum**. Кстати

говоря, размеры списков **x_cum [1:]** и **C_cum** будут совпадать, потому что мы отбрасываем нулевой элемент в первом списке **x_cum**.

Мы выводим график не только прямыми отрезками, но используем маркеры-кружочки: **marker='o'**.

Затем мы построим теоретические значения интегральной функции нормального распределения **C**. Для этого нам понадобится функция **norm.cdf**. Сокращение **cdf** расшифровывается как **Cumulative distribution function** — «кумулятивная (интегральная) функция распределения». В качестве параметров мы передадим

— **xx** — ранее построенный список значений «иксов»;
— **loc=mu** — математическое ожидание;
— **scale=sigma** — стандартное отклонение.

2. Анализ текстовых данных

2.1. ОБРАБОТКА ТЕКСТОВ НА ЕСТЕСТВЕННОМ ЯЗЫКЕ

При анализе текстов используют разные технологии. В качестве примера мы рассмотрим частотный анализ текстов. Это подсчёт числа появления слов в выбранном тексте. Иногда для такого метода используется выражение **bag of words** — «мешок слов».

Задание. Просмотрите в Википедии статью **Мешок слов**.

Нам предстоит построить гистограмму частот и облако слов (тегов). При выполнении лабораторной работы можно будет убедиться в том, что облако слов представляет собой визуальный аналог гистограммы. Это один из простых инструментов для оперативного анализа текстов больших объемов на предмет семантики / тематического содержания.

Задание. Просмотрите в Википедии статью **Облако тегов**.

В качестве исходных данных для частотного анализа мы используем автоматическую расшифровку (транскрибирование, субтитры) видеороликов **Youtube**. Аналогично можно проводить анализ выгрузки истории чатов в популярных мессенджерах типа **Whatsapp**, **Telegram**, **Discord**.

Таким образом, мы получаем в наше распоряжение большое количество разнообразных открытых источников неструктурированных текстовых данных большого объёма. В результате каждый студент получает индивидуальный вариант заданий.

Работа по анализу данных выполняется на облачной платформе **Google Colab**. Студенты смогут выполнять работу на любом компьютере, подключенном к интернет. Из программного обеспечения на локальном компьютере требуется только брау-

зер. Также при обработке данных будет задействован табличный редактор типа **MS Excel** либо другой, совместимый по формату файлов, например, **Libre Office Calc** или **Google Sheets**.

Одно из направлений в технологиях анализа больших данных — это обработка текстов на естественном языке, то есть текстов, написанных живыми людьми. Соответствующее английское название: **Natural Language Processing** (**NLP**) — не путать с нейролингвистическим программированием.

Большое количество текстов доступно в оцифрованном виде — в сети предприятия либо в интернет. Однако, такие тексты, как правило, слабо структурированы и не предназначены для машинной обработки.

Один из подходов к анализу текстов на естественном языке — это изучение частоты появления в тексте различных слов. Наиболее часто встречающиеся слова могут иметь отношение к теме выступления или лекции, что позволяет в автоматизированном режиме выделить ключевые моменты содержания. Проще говоря, можно составить список так называемых **КЛЮЧЕВЫХ СЛОВ**.

При частотном анализе текста приходится попутно решать ряд дополнительных проблем. Отметим две основные трудности: обилие служебных слов и грамматическое/морфологическое разнообразие словоформ.

Во-первых, в текстах чаще всего встречаются **служебные слова**, не несущие смысла анализируемого сообщения. Это предлоги, местоимения, числительные и некоторые глаголы. Для фильтрации таких «лишних» слов требуется составлять вспомогательный список так называемых «стоп-слов». Причём этот список может существенно различаться в зависимости от категории анализируемого текста.

Во-вторых, в русском языке имеется достаточное количество различных **форм** одного и того же слова. Существительное: род, число, падеж. Глагол: род, вид, время, спряжение. Аналогичная картина наблюдается с местоимениями, прилагательными и на-

речиями. Здесь требуется привести каждое слово в первоначальную / словарную форму.

Кроме того, автоматическое транскрибирование (перевод аудиозаписи в текст) вносит в текст искажения за счёт неидеального распознавания устной речи. В расшифровке видео появляются ошибки, замена одних слов на другие, даже на слова из другого языка. Не следует забывать о жаргоне и ошибках в текстовых сообщениях в чатах, в том числе и намеренных (так называемый «олбанский язык»). В ряде случаев ошибки могут создавать дополнительный информационный шум. В некоторых ситуациях, наоборот, именно ошибки и служебные слова являются ценным источником информации.

2.2. ОБЛАКО СЛОВ (ТЕГОВ)

Познакомимся с таким инструментом, как **облако слов**. Вначале нужно подключить необходимые библиотеки, см. рис. В рамках одного блокнота **Jupyter Notebook** можно загружать одни и те же библиотеки несколько раз. Это может быть полезно, если мы запускаем отдельные ячейки блокнота.

Дело в том, что после отключения от облачного блокнота все предыдущие результаты теряются. А при очередном подключении к блокноту мы можем запустить несколько ячеек, и не обязательно в правильном порядке.

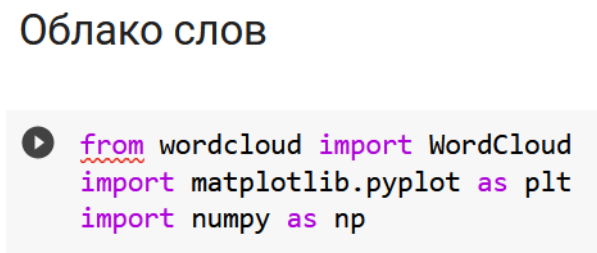


Рис. Загружаем библиотеки

Сформируем несколько строк текста, см. рис. Мы используем тройные кавычки. Это означает, что мы создаём переменную типа **string** — «строка». Но эта «строка» состоит из нескольких строк. Такой объект обычно называют **Python Multiline String** — «Многострочная строка Питона». Здесь **string** — это тип переменной (фактически список или массив символов), а **line** — это строка текста. Оба слова переводятся как «строка».

```
text = """Statistics
Machine Learning
Big Data
Deep Learning
Neural Nets
Regression
Classification
Clustering
Dimension Reduction
Professor Arkov"""
```

Рис. Многострочная строка

Теперь построим наше облако слов. Создаем вспомогательный графический объект — переменную под названием **wordcloud** из нашей многострочной строки.

Затем выводим это объект на экран с помощью функции **plt.imshow**.

```
wordcloud = WordCloud().generate(text)
plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```



Рис. Первое облако слов

Настроим наше изображение, см. рис. Указываем следующие параметры:

— **width=1000** — ширина изображения (в пикселах);

— **height=600** — высота изображения;

— **background_color='white'** — цвет фона — белый.

Отключаем вывод осей координат командой **plt.axis («off»)**.

Наконец, отключаем вывод служебной информации о графике: **plt.show ()**.

```
wordcloud = WordCloud(width=1000, height=600,
                      background_color='white').generate(text)
plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```



Рис. Облако на белом фоне

2.3. МАСКА

Следующий шаг в «украшательстве» облака — придать графику любую форму. Создаём маску в любом графическом редакторе. Например, запустим **Paint**. Маска — это белый фон и черная фигура, которая будет заполнена нашими словами. Маска придает облаку любую форму.

На белом фоне рисуем овал и заливаем его черным цветом. Сохраняем в файл под простым названием, лучше латинскими буквами.

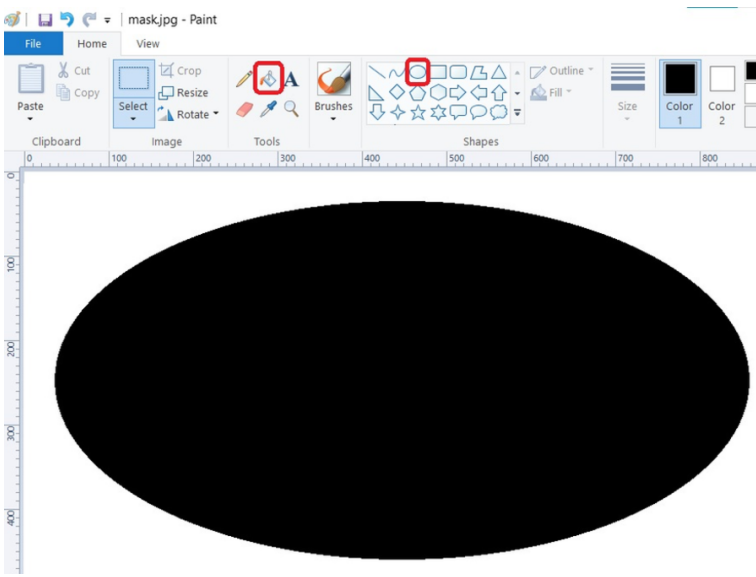


Рис. Создаем маску

Подключим библиотеки и загрузим наш файл маски. На этот

раз задействуем другой приём — используем функцию **files.upload**, см. рис.

Появляется кнопка Browse. Нажимаем кнопку и выбираем нужный файл.

После успешной загрузки мы увидим сообщение с указанием названия и размера файла.

Файл также выводится в разделе **Files** — после нажатия кнопки **Refresh — Обновить**.

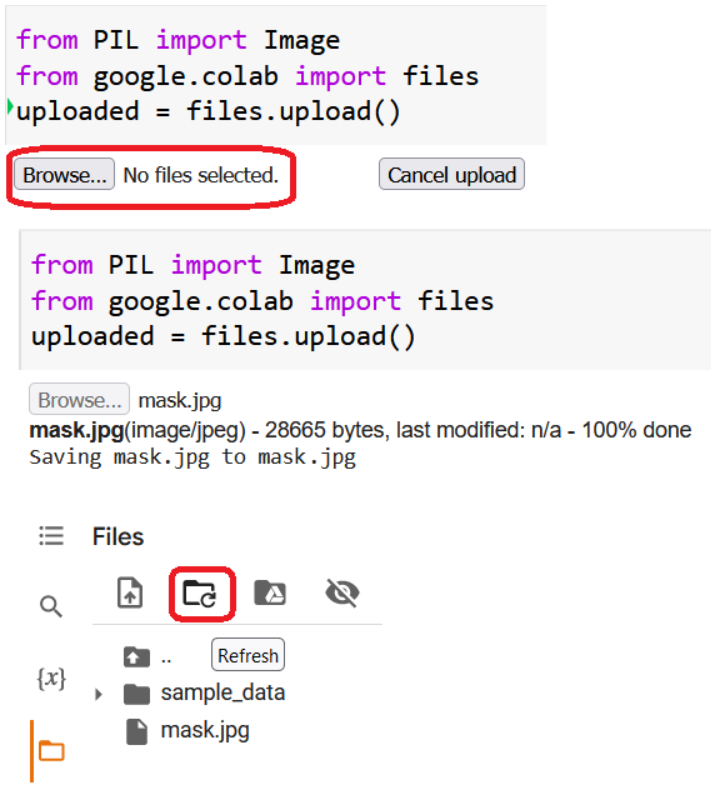


Рис. Загружаем файл маски

Файл загружен. Вводим команду **ls** — сокращение от английского **LIST** — список файлов. Это команда операционной системы Linux, которая запущена в облаке на нашей виртуальной машине. Команды ОС мы вводим в кодовой ячейке после восклицательного знака.

Чтобы использовать маску, нужно прочитать содержимое файла командой **Image. open**. Преобразуем прочитанные данные в массив **Numpy** с помощью команды **np.array.** Поместим полученный массив в переменную **oval_mask**.

Мы сразу же можем уточнить, какой тип имеет наша переменная с помощью команды **type**. Как видим, это массив **Numpy**.

```
!ls
```

```
mask.jpg  sample_data
```

```
oval_mask = np.array(Image.open("mask.jpg"))
```

```
type(oval_mask)
```

```
numpy.ndarray
```

Рис. Читаем файл маски

Теперь мы можем сформировать облако слов в форме нашей маски, см. рис. Для этого указываем параметр **mask=oval_mask**. В остальном все точно так же, как и в предыдущем примере. Как видим, облако приобрело нужную форму.

```
wordcloud3 = WordCloud(width=1000, height=600,
                       background_color='white',
                       mask=oval_mask).generate(text)
plt.imshow(wordcloud3)
plt.axis("off")
plt.show()
```



Рис. Облако слов по маске

Наконец, сохраним наше художественно оформленное облако в файл, см. рис. Используем функцию **wordcloud3.to_file** и указываем имя файла — в кавычках.

Файл появляется в списке файлов в текущем каталоге.

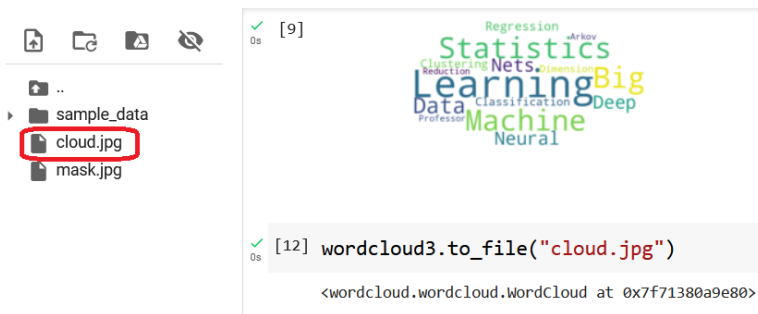


Рис. Сохраняем облако слов в файл

Пока что наш файл лежит на удаленной виртуальной ма-

шине — где-то в интернете. Он сохраняется только на время текущего сеанса работы. При отключении от блокнота мы этот файл потеряем.

Чтобы скачать файл на локальную машину, наводим курсор на файл и нажимаем на «многоточие», см. рис. В выпадающем меню выбираем пункт Download — скачать.

Находим файл в каталоге загрузок и любуемся нашим облаком с помощью любой программы просмотра изображений.

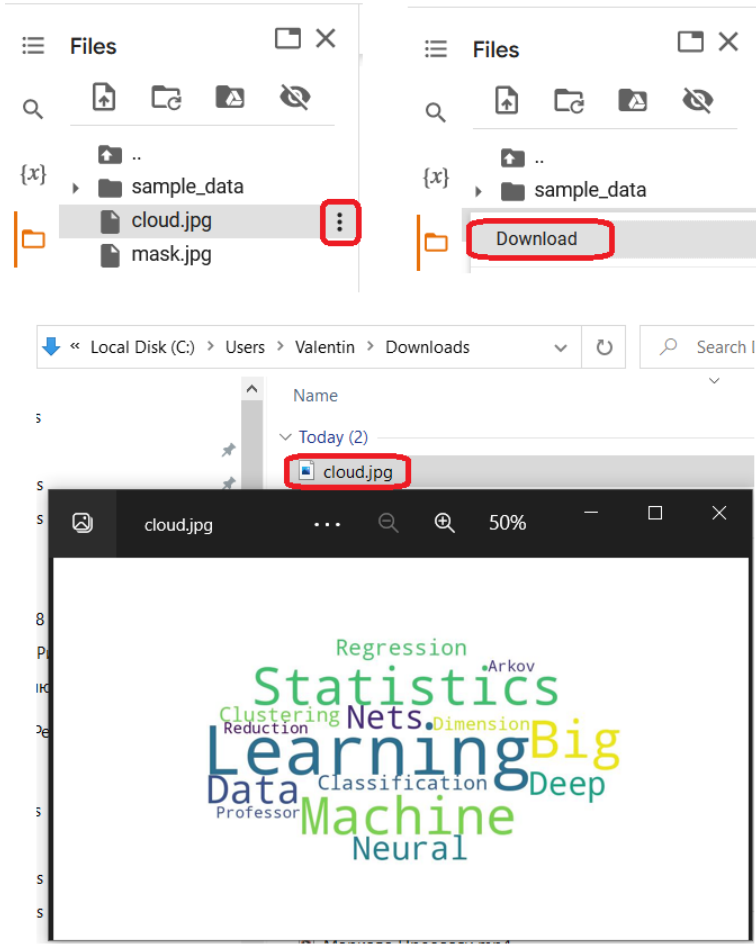


Рис. Скачиваем файл на компьютер

2.4. ОБЛАКО ФРАЗ

Хотя мы старательно разбили текст на строки, при построении облака слов были использованы отдельные слова, а не фразы. С этим тоже можно поработать.

Соединим слова в наших выражениях с помощью символа подчёркивания: **Machine_Learning**, см рис. Теперь эта цепочка символов будет рассматриваться как одно «слово».

```
text = """Statistics
Machine_Learning
Big_Data
Deep_Learning
Neural_Nets
Regression
Classification
Clustering
Dimension_Reduction
Professor_Arkov"""
```

Рис. Объединение слов

Повторим построение изображения с маской, см. рис.

Теперь наше облако построено не из отдельных слов, а из фраз. Конечно, символ подчеркивания смотрится не слишком привлекательно. Это было решение задачи, что называется, «в лоб».

Более грамотное решение можно предложить, если использовать так называемые **би-граммы — Collocations / Bigrams**:

https://amueller.github.io/word_cloud/generated/wordcloud.WordCloud.html[1]

Это словосочетания, состоящие из двух слов.

```
wordcloud3 = WordCloud(width=2000, height=1200,
                       background_color='white',
                       mask=oval_mask).generate(text)
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud3)
plt.axis("off")
plt.show()
```



Рис. Облако из фраз

Попутно зададим размер изображения на экране побольше — с помощью параметра **figsize** функции **plt.figure**. Но это размеры картинки на экране. А вот размеры изображения, которое мы сможем скачать на компьютер, заданы параметрами **width** и **height** при вызове функции **WordCloud**. Правда, здесь участвует маска, а это файл с другими размерами — в пикселах. В конечном счете, нас интересует качество полученного изобра-

[1] https://ridero.ru/link/DHQWdJKUBxNeXlXLFz8vR

жения, подробная передача деталей — а это зависит от количества точек (пикселов).

Проверим, какие размеры мы получили на самом деле. В **Проводнике Windows** вызываем **Properties — Details** (**Свойства — Подробности**), см. рис.

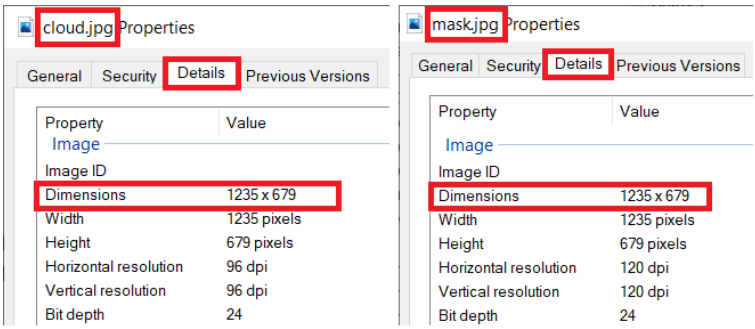


Рис. Размеры изображений

Задание. Проверьте размеры изображения и маски с помощью **Проводника**. Измените размеры маски и проверьте, как изменились размеры изображения облака. Проверьте размеры маски в Python с помощью свойства объекта **oval_mask.shape**.

2.5. СЛОВАРЬ ВЕСОВ (ЧАСТОТ)

Идём дальше и зададим разный «вес» нашим словам и выражениям. Для этого нам понадобится объект Python под названием «словарь» — **dictionary**, см. рис. Словарь содержит ключи и значения, соответствующие этим ключам.

Словарь создают с помощью фигурных скобок. Внутри этих скобок мы даём пары «ключ-значение» через двоеточие.

Ключами у нас будут слова, а значениями — вес этих слов. Более важные слова должны изображаться более крупным шрифтом.

```
dict_cloud = {'Statistics':20, 'Machine Learning':15,
              'Big Data':8, 'Deep Learning':7,
              'Neural Nets':6, 'Regression':5,
              'Classification':4, 'Clustering':3,
              'Dimension Reduction':2, 'Professor Arkov':1}
```

```
dict_cloud
```

```
{'Statistics': 20,
 'Machine Learning': 15,
 'Big Data': 8,
 'Deep Learning': 7,
 'Neural Nets': 6,
 'Regression': 5,
 'Classification': 4,
 'Clustering': 3,
 'Dimension Reduction': 2,
 'Professor Arkov': 1}
```

Рис. Создаем словарь для построения облака

Повторим построение облака и создадим очередной объект

wordcloud4, см. рис. Действительно, буквы на изображении разного размера.

```
wordcloud4 = WordCloud(width=1000, height=600,
                       background_color='white',
                   mask=oval_mask).generate_from_frequencies(dict_cloud)
plt.figure(figsize=(18, 8))
plt.imshow(wordcloud4)
plt.axis("off")
plt.show()
```



Рис. Облако слов по словарю

Сохраним изображение в файл и скачаем его на локальный компьютер, см. рис. Здесь мы используем новую функцию (метод загрузки объектов) **download** из модуля **files**.

```
wordcloud4.to_file("dict_cloud.jpg")
```

```
<wordcloud.wordcloud.WordCloud at 0x7fe2d2b4dc40>
```

```
!ls
```

```
cloud.jpg  dict_cloud.jpg  mask.jpg  sample_data
```

```
files.download("dict_cloud.jpg")
```

```
Downloading "dict_cloud.jpg":
```

Рис. Сохраняем и загружаем изображение

Мы потренировались в работе с английскими фразами. Пора перейти к русским. Сформируем «словарь» с весовыми коэффициентами для построения облака слов, см. рис.

```
dictRu = {'Статистика':20,
          'Машинное обучение':15,
          'Большие данные':8,
          'Глубокое обучение':7,
          'Нейронные сети':6,
          'Регрессия':5,
          'Классификация':4,
          'Кластеризация':3,
          'Понижение размерности':2,
          'профессор Арьков':1}
```

Рис. Русский «словарь» с весами

Сгенерируем и выведем на экран наше облако, используя

уже известные нам команды, см. рис.

```
wordcloud5 = WordCloud(width=1000,
                       height=600,
                       background_color='white',
                       mask=oval_mask).generate_from_frequencies(dictRu)
plt.figure(figsize=(18, 8))
plt.imshow(wordcloud5)
plt.axis("off")
plt.show()
```



Рис. Облако слов на русском

2.6. РАСШИФРОВКА ВИДЕО YOUTUBE

При анализе аудио- и видеозаписей требуется вначале получить текстовый файл с расшифровкой записи. Для перевода звуковых записей в текст используют сервисы транскрибирования (**Speech to Text**). В том числе, модные нынче нейронные сети.

Есть один сервис расшифровки — экзотический, зато совершенно бесплатный. Запускаем звуковую запись на воспроизведение и отправляем этот аудиопоток голосовому помощнику. Это самый долгий путь, потому что придётся дождаться, пока проиграется вся запись.

Бесплатный сервис транскрибирования можно организовать с помощью личного канала **Youtube**. Загружаем видеоролик на свой канал, и через несколько часов можно будет получить расшифровку записи в виде субтитров. Свой ролик можно опубликовать с ограничением доступа — только по ссылке. В этом случае кроме нас никто даже не увидит данную запись. Тем не менее, автоматическая расшифровка будет сделана.

Скачать субтитры **Youtube** можно самыми разными способами. Мы рассмотрим три простых варианта:

— страница демонстрации ролика **Youtube**;

— сторонний сервис скачивания субтитров;

— творческая студия **Youtube**.

В качестве примера мы получим расшифровку лекции автора данной работы по дисциплине «Бизнес-аналитика и статистика». Ссылка на видео:

https://youtu.be/kBiD09CATeo

2.6.1. СТРАНИЦА ВИДЕОРОЛИКА

Самое простое решение — скачать расшифровку на странице видеоролика. Нажимаем кнопку [...] под видео и выбираем пункт **Посмотреть расшифровку видео**, см. рис.

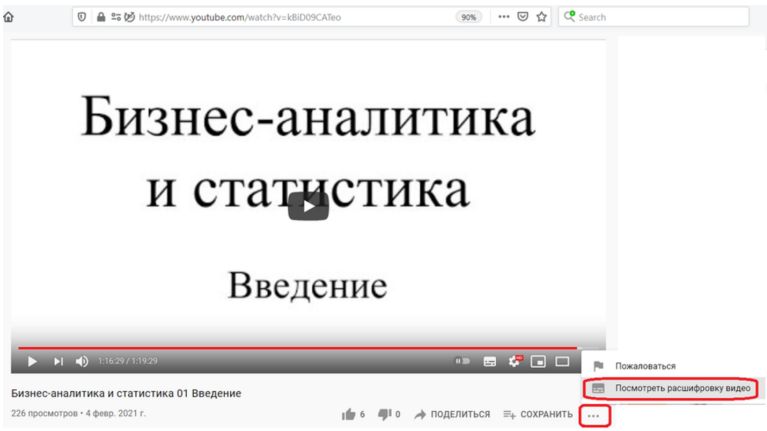


Рис. Просмотр расшифровки (субтитров)

В правой части окна браузера выводится текст расшифровки. Нажимаем кнопку «вертикального многоточия» над расшифровкой в панели субтитров и отключаем метки времени: **Показать или скрыть временные метки**, см. рис.

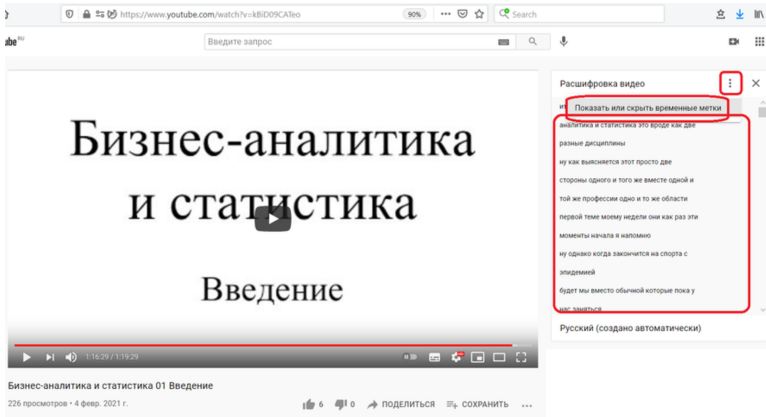


Рис. Копирование текста расшифровки

Выделяем с помощью мыши интересующий текст. На выделение и прокрутку текста может потребоваться несколько секунд. Затем вставляем текст из буфера обмена в Блокнот.

Можно поступить по-другому. Выделяем весь текст на странице: **[Crtl + A]**. Копируем в буфер: **[Ctrl + C].** Вставляем в **Блокнот**: **[Ctrl + V]**. Затем удаляем всё лишнее.

Сохраняем текстовый файл, см. рис. В файле находится только текст, ничего лишнего. Никаких меток, никаких знаков препинания.

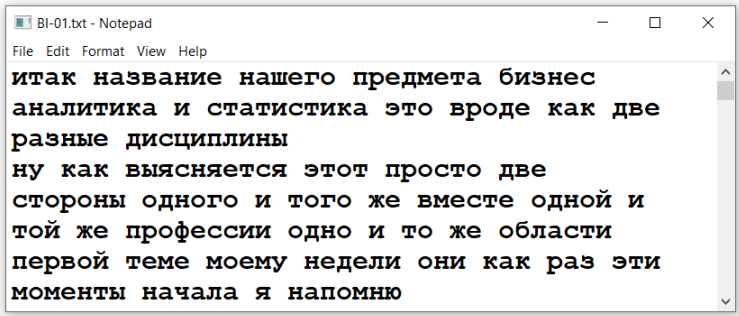


Рис. Текст расшифровки в файле

2.6.2. СУБТИТРЫ DOWNSUB

Второй вариант скачивания субтитров — сервис **DownSub**: https://downsub.com/

Переходим на указанный сайт, вводим адрес видео и нажимаем кнопку **Download**. Выбираем текстовый вариант без временных меток: **TXT**, см. рис.

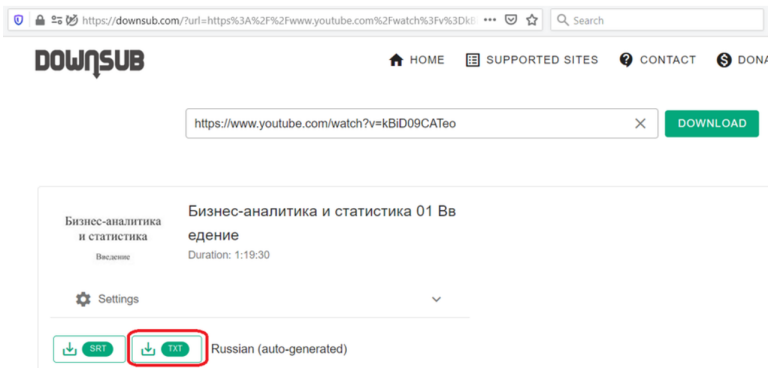


Рис. Скачивание текста субтитров

Получаем текстовый файл и изучаем содержимое в **Блокноте**, см. рис. Всё хорошо, но появились лишние пустые строки. Придётся дополнительно удалять пустые строки с помощью функции поиска и замены.

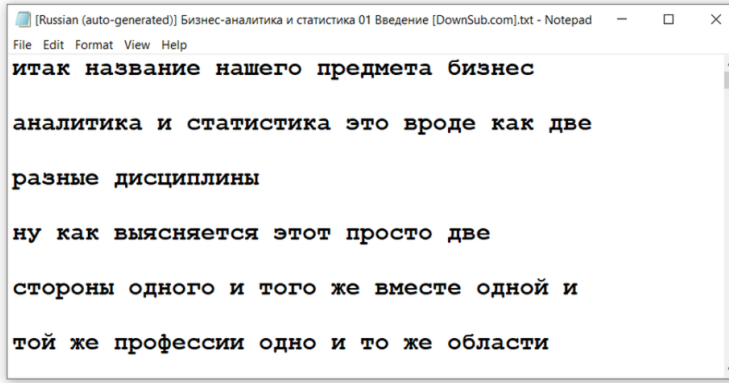


Рис. Текстовые субтитры**DownSub**

2.6.3. СТУДИЯ YOUTUBE

Конечно, можно скачать субтитры напрямую из **Творческой Студии Youtube**. Для этого выбираем видео, находим раздел **Язык и субтитры**, выбираем автоматически созданные субтитры и нажимаем **Скачать**, см. рис.

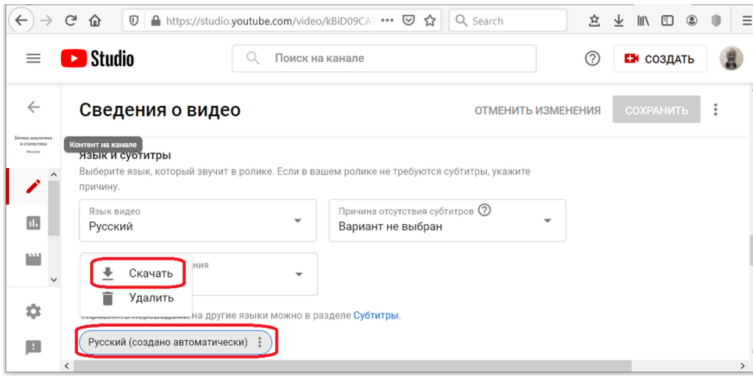


Рис. Скачивание субтитров**Студии Youtube**

Получаем текстовый файл, но с временными метками, см. рис. Удаление временных меток — это лишняя работа, и мы её в данной работе не рассматриваем. Такой полноценный файл субтитров решает совсем другие задачи, выходящие за рамки текущего исследования.

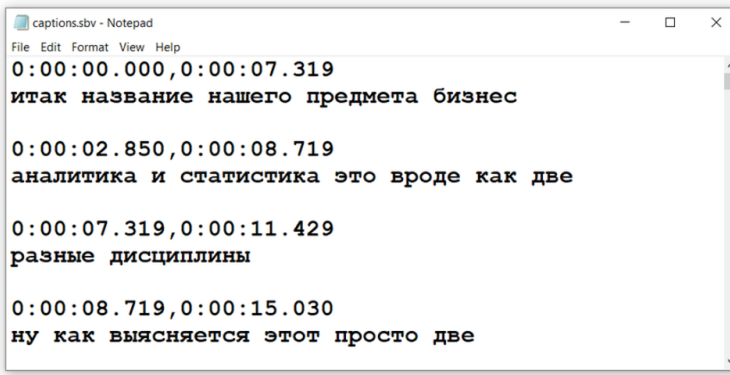


Рис. Файл субтитров**Youtube**

Полученные субтитры, конечно же, не идеальные. Результаты автоматической расшифровки зависят от качества записи и разборчивости произношения докладчика. Но для первоначального анализа материала этого вполне достаточно. Мы сможем просмотреть текст полуторачасового выступления за пару минут. Затем с помощью частотного анализа текста мы сможем выделить ключевые слова, которые чаще всего встречаются в записи. И этот список ключевых слов представляет собой сжатое содержание, смысл материала.

2.7. ПРЕДВАРИТЕЛЬНЫЙ АНАЛИЗ ТЕКСТА

Итак, мы скачали расшифровку видео — любым способом, как описано выше. Переходим к работе с текстом в Python.

Переходим к работе с файлом. Вначале нужно загрузить текстовый файл с локального компьютера на облачный диск. Мы эту процедуру уже обсудили выше. Повторяться не будем.

Открываем наш файл на чтение и читаем его содержимое в переменную **t0**. Можно скопировать путь к файлу и вставить его в команду, как показано на рис.

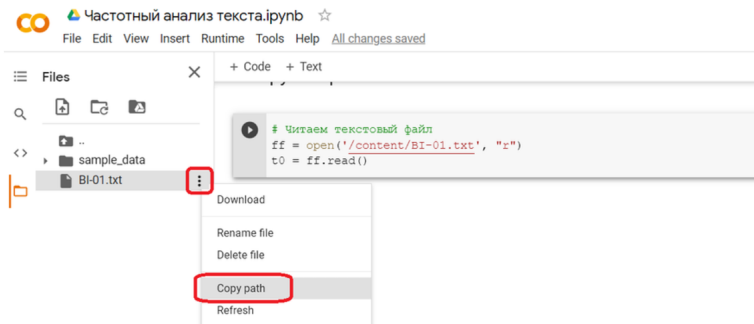


Рис. Чтение файла

Содержимое текстового файла теперь находится в переменной-срок **t0**. Выводим переменную на экран, см. рис. Можно видеть, что прочитан не только текст, но и разделители строк — знакомая комбинация символов **\n**, которая используется в команде **printf** в языке Си.

```
t0
```

```
'итак название нашего предмета бизнес аналитика и статистика это вроде как две
е вместе одной и\nтой же профессии одно и то же области первой теме моему неде
порта с\nэпидемией будет мы вместо обычной которые пока у нас заняться\nну к с
е люди вопросам будем хорошо у нас есть 10 указано три адреса интернете на сам
риалы ссылки объявления обряды проходите то чтобы в конце семестра\nне читали
что я буду дублировать на сайте орков narod.ru просто на случай выстаивать вре
ей мир именно та штука но и работать\nмедленнее или часть затяжка\nзум ...'
```

Рис. Содержимое переменной

Определим размер полученной переменный. Сравним с результатами подсчета количества слов и символов в текстовом редакторе MS Word, нажав на соответствующую кнопку в нижней части окна текстового редактора, см. рис.

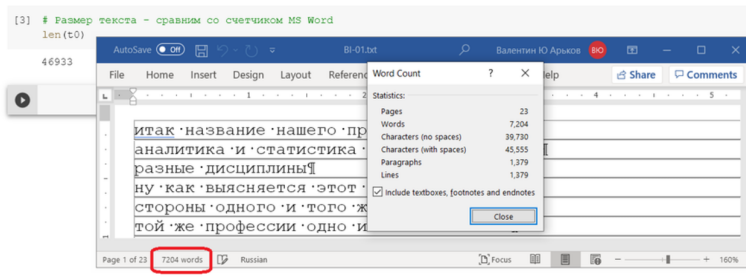


Рис. Количество слов и символов

Выясним, какая кодировка используется в нашем текстовом файле. Откроем файл в текстовом редакторе **Notepad**. В строке состояния видим указание на кодировку **UTF-8**, см. рис.

Задание. Выясните, сколько байт используется в **UTF-8** для хранения одного символа. Для этого рекомендуем ознакомиться с соответствующей статьёй в Википедии:

https://ru.wikipedia.org/wiki/UTF-8

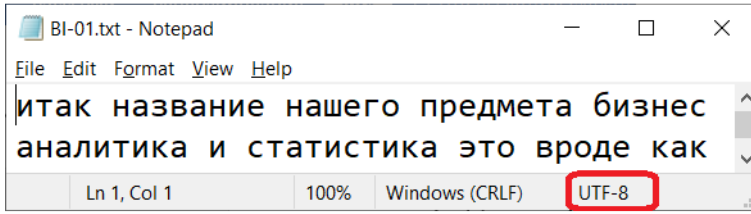


Рис. Кодировка файла

Для определения количества прочитанной информации мы использовали команду **len**. Разберемся, что делает эта команда. Скорее всего, это сокращение от английского слова **lenght**, то есть «длина». Чтобы получить описание команды, пишем название команды и вопросительный знак, см. рис.

В правой части окна читаем описание: «количество элементов в контейнере». Проще говоря, это размер массива или длина строки.

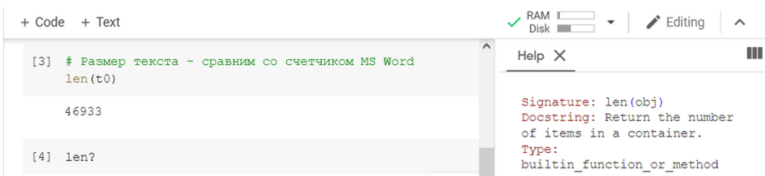


Рис. Описание команды **len**

Откроем файл в редакторе **Notepad++** и вызовем в верхнем меню пункт **View — Summary**. Получаем сводные данные по нашему документу. Здесь указано количество знаков без учёта символа конца строки — **Characters without line endings**, а также число слов — **Words** и строк — **Lines**, см. рис. Сравним со счётчиком **MS Word**.

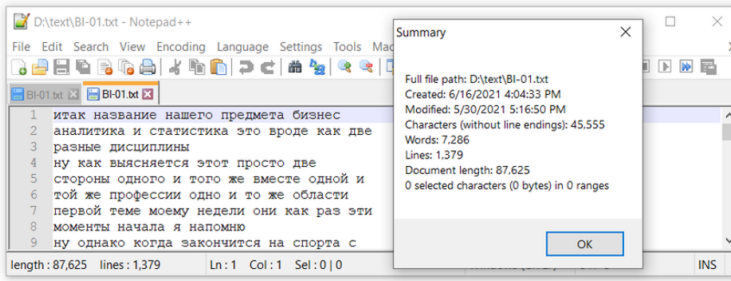


Рис. Сводные данные по документу

Теперь проверим, что нам сообщит операционная система. Вводим восклицательный знак и команду **LS** — это команда Linux — сокращение от **LIST**, то есть «список файлов». В нашем случае получим подробную информацию об указанном файле. Операционная система сообщает размер файла в байтах, см. рис.

Если навести курсор на наш файл в левой части окна, появляется всплывающая подсказка. Здесь размер файла измеряется в килобайтах.

Чтобы перевести байты в килобайты, поделим число байт на 1024, см. рис.

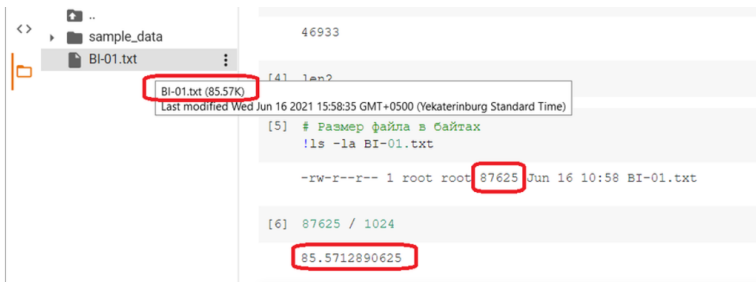


Рис. Размер файла

Выведем на экран первые 100 элементов переменной **t0**, см. рис. В квадратных скобках мы указываем номера начального и конечного элементов списка, причем нумерация начинается с нуля, а последний элемент не выводится. Если первый номер не указан, вывод начинается с нулевого элемента. Если конечный номер 100, то последним выводится элемент номер 99. Таким образом, мы получаем элементы строки с номерами от 0 до 99, итого 100 символов.

Рассмотрим полученные результаты. Как мы уже говорили, на экран выводится, в том числе, и символ перевода строки **\n**. Кроме того, на экране дополнительно выводятся кавычки в начале и в конце вывода. Таким образом, переменная **t0** — это просто текстовая строка. Весь текстовый файл был прочитан в переменную в виде одной строки большой длины.

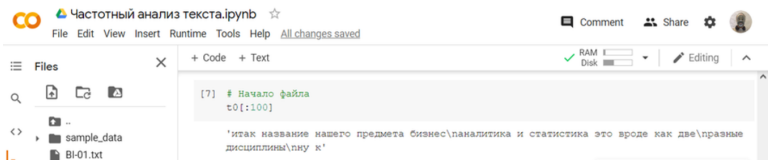


Рис. Начало файла

Для дальнейшей работы нам необходимо удалить символ конца строки. Заменим его на пробел с помощью функции **replace**, см. рис.

Выведем на экран первые 100 элементов новой переменой **t1**. Теперь это строка стала удобочитаемой. Слова разделены пробелами. Длина строки не изменилась.

Определим тип переменной с помощью команды **type**. Нам сообщают, что это строка: **str** — от английского слова **STRING**.

```
[7] # Начало файла
    t0[:100]

    'итак название нашего предмета бизнес\паналитика и статистика это вроде как две\nразные д
    исциплины\nну к'

[8] # Заменим символ перевода строки на пробел
    t1 = t0.replace('\n',' ')
    t1[:100]

    'итак название нашего предмета бизнес аналитика и статистика это вроде как две разные дис
    циплины ну к'

[9] len(t1)

    46933

[10] type(t0)

    str
```

Рис. Удаление символа конца строки

2.8. ОБЛАКО СЛОВ

Итак, мы получили длинную строку текста, в которой слова разделены пробелами. Построим по этой строке облако слов «в лоб», безо всякой обработки.

Создадим новый раздел. Загружаем необходимые библиотеки. Строим облако слов по переменной **t1**.

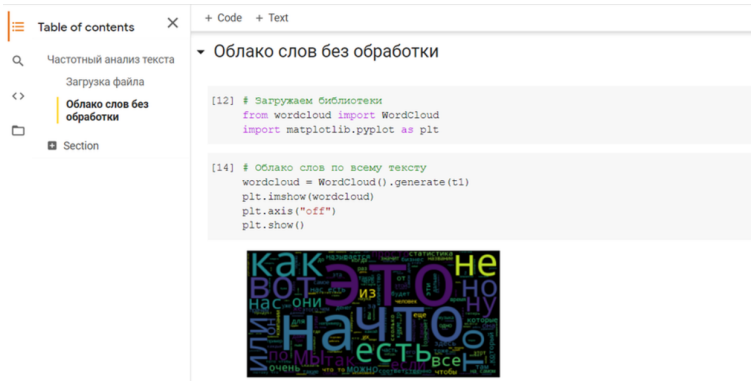


Рис. Облако слов без обработки

Настроим оформление графика, как мы уже проделали выше. Выбираем белый фон — **background_color**.

Задаем размеры облака слов в точках — **width**, **height**.

Укажем размеры графика **figsize** в дюймах, см. рис.

```
# Белый фон, большие размеры
wordcloud = WordCloud(width=2000, height=1000, background_color='white').generate(t1)
plt.figure(figsize=(16,8))
plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```



Рис. Настройка оформления

Рассмотрим полученный график. Большими буквами выделены самые обычные слова: «на», «что», «это». Причина проста — эти слова встречаются в нашем тексте чаще всего. Как мы помним, чем выше частота, тем больше размер шрифта на изображении.

Найдём частоту для каждого слова. Для этого разобьём нашу длинную строку на отдельные слова. В качестве разделителя здесь выступает символ пробела, см. рис.

```
t2 = t1.split(" ")
t2[:10]
```

```
['итак',
 'название',
 'нашего',
 'предмета',
 'бизнес',
 'аналитика',
 'и',
 'статистика',
 'это',
 'вроде']
```

Рис. Разбиваем строку на слова

Загрузим библиотеку **pandas** для анализа табличных данных. Создадим объект типа **DataFrame** и посчитаем частоту каждого значения с помощью функции **value_counts**, см. рис.

Мы получили результаты в две колонки: сами слова и их частоты. Как видим, в этом тексте чаще всего встречаются общеупотребительные слова: «и», «это», «на», «что». Это так называемые служебные слова. В первой десятке нет ни одного слова, несущего смысл текста. В дальнейшем нам предстоит избавиться от таких **служебных слов**, которые не несут особой смысловой нагрузки.

```
import pandas as pd
x1 = pd.DataFrame(t2)
freq = x1.value_counts()
freq[:10]
```

```
и       224
это     149
на      125
что     118
в        92
как      91
вот      84
не       71
есть     70
нас      67
dtype: int64
```

Рис. Частота слов

Задание. Просмотрите в Википедии статью **Служебные слова**.

Теперь из этих результатов сформируем новый объект типа **DataFrame**. Переименуем столбец частот и дадим ему название **Frequency**, см. рис.

```
x2 = pd.DataFrame(freq, columns=["Frequency"])
```

```
x2
```

	Frequency
0	
и	224
это	149
на	125
что	118
в	92
...	...
любите	1
лучшая	1
лучи	1
лозы	1
ящук	1

2694 rows × 1 columns

Рис. Таблица частот

Первоначально слова были использованы в качестве индекса. Выделим колонку слов в отдельный столбец с помощью функции **reset_index**. С помощью функции **rename** переименуем этот столбец **Word**, см. рис. Здесь использован параметр **inplace** с значением **True**. Это позволяет сразу же внести изменения с существующую таблицу, не используя операцию присваивания.

```
x3 = x2.reset_index()
x3.rename(columns={0: "Word"}, inplace=True)
x3
```

	Word	Frequency
0	и	224
1	это	149
2	на	125
3	что	118
4	в	92
...	...	...
2689	любите	1
2690	лучшая	1
2691	лучи	1
2692	лозы	1
2693	ящук	1

2694 rows × 2 columns

Рис. Столбец слов

Теперь разберемся с тем, какие слова были подсчитаны в нашем примере.

Отсортируем наши слова. Расположим их в алфавитном порядке с помощью функции **sort_values**.

Выведем на экран последние семь строк в нашей таблице функцией **tail**. Английское слово **TAIL** означает «хвост животного», а также «хвост поезда или чего-то подобного». В нашем случае это «последние строки таблицы».

Как видим, в этом списке присутствует одно и тоже слово «язык», но в разных формах, в разных падежах. В дальнейшем нам нужно будет привести слова к начальной форме. Тогда мы сможем грамотно подсчитать, сколько раз в тексте употреблялось каждое слово (в любой грамматической форме).

```
x4 = x3.sort_values(by="Word")
x4.tail(7)
```

	Word	Frequency
733	явно	2
732	язык	2
1243	языке	1
247	языке	4
731	языком	2
1242	язычки	1
2693	ящук	1

Рис. Сортировка слов

У нас есть еще одна проблема с анализом текстов, и мы сейчас её продемонстрируем. Выделяем собранные слова в один массив с помощью функции **unique**. Обратим внимание, что мы используем столбец **Word** и для этого указываем название столбца через точку после названия таблицы. Выведем на экран первые 200 слов, см. рис.

Как видим, здесь есть английские слова, а также цифры и знаки препинания. Есть здесь и комментарии из субтитров: **[музыка]**.

Это отдельная проблема в обработке текстов, и от этих «лишних» элементов тоже предстоит избавиться.

```
x5 = x4.Word.unique()
x5[:100]
```

```
array(['', ',', '.', '1', '1-го', '10', '11', '2', '2-ой', '20', '239',
       '3', '30', '36', '37', '4', '4-го', '5', '50', '500', '6', '70',
       '9000', '[музыка]', 'ap', 'avanti', 'awards', 'b', 'be', 'bi',
       'business', 'call', 'cd', 'choco', 'cicada', 'classic', 'color',
       'come', 'cotto', 'dark', 'deity', 'dirol', 'distribution',
       'fashion', 'ford', 'freely', 'he', 'intelligence', 'iso', 'like',
       'microsoft', 'narod.ru', 'nuxe', 'of', 'omega', 'orbea', 'others',
       'pie', 'power', 'slow', 'teen', 'this', 'to', 'touch', 'up',
       'white', 'world', 'xi', 'xiii', 'z', 'а', 'абсурда', 'августа',
       'авто', 'автоинспекция', 'автомобиль', 'автомобиля', 'авторы',
       'агентов', 'агентство', 'агрегирование', 'агрегирования', 'адрес',
       'адреса', 'аккуратно', 'акра', 'акта', 'активно', 'алфавит',
       'алфавите', 'альтернативное', 'альфа', 'альфа-самец',
       'американские', 'американский', 'американских', 'американской',
       'америке', 'анализ', 'анализа'], dtype=object)
```

Рис. Уникальные значения

Теперь попробуем оставить в исходной строке только русские буквы, а также пробел и черточку (дефис). Сформируем строку из «желательных» символов и назовём её **alpha**, см. рис.

Здесь мы используем функцию (метод) **join**. Этот инструмент позволяет объединить список в одну строку. Перед ним указаны пустые кавычки. Это строка, которая беде использована как разделитель при объединении строк. В нашем примере разделителя не будет.

В скобках идёт любопытная конструкция. Фактически, здесь использован цикл for, внутри которого проверяется условие принадлежности каждого символа к списку **alpha**. Если условие выполняется, мы возвращаем очередной символ.

```
# Оставим только буквы, пробелы и чёрточки
alpha = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя -"
t5=''.join(x for x in t1 if x in alpha)
t5
```

```
'итак название нашего предмета бизнес аналитика и статистика это вроде как две р
вместе одной и той же профессии одно и то же области первой теме моему недели он
с эпидемией будет мы вместо обычной которые пока у нас заняться ну к сожалению п
просам будем хорошо у нас есть  указано три адреса интернете на самой адрес эта
бъявления обряды проходите то чтобы в конце семестра не читали какие-то остатки
вать на сайте орков  просто на случай выстаивать временно не будет работать сайт
работать медленнее или часть затяжка зум здесь и на начнем с конца че...'
```

Рис. Только буквы

Рассмотрим выражение в скобках поподробнее, см. рис. Присвоим это выражение как значение переменной **c**. Выводим на экран тип переменной **c**. Нам сообщают, что это «генератор» — конструкция Питона, которая выдаёт очередное значение «по требованию».

Выводим на экран первые 10 элементов **c**. Чтобы получить значения элементов c, мы преобразуем её в список — **list**. Далее в квадратных скобках мы указали, что нам нужны первые десять элементов этого объекта.

Обратим внимание, что вывод на экран в середине ячейки требует использования функции **print**. Если в последней строке ячейки ввести только имя переменной, она будет выведена на экран даже без **print**.

```
c = (x for x in t1 if x in alpha)
print(type(c))
list(c)[:10]
```

```
<class 'generator'>
['и', 'т', 'а', 'к', ' ', 'н', 'а', 'з', 'в', 'а']
```

Рис. Генератор

Повторим предыдущие шаги по обработке текста и изучим полученные результаты. Мы использовали команду split, чтобы разделить строку на отдельные слова. В качестве мы указали символ пробела. Теперь у нас остались только русские слова. В некоторых словах сохранился дефис, см. рис.

Таким образом, мы избавились от английских слов, а также от цифр и знаков препинания. Обратим внимание на то, что в расшифровке видео используются только символы нижнего регистра.

```
t6 = t5.split(" ")
x6 = pd.DataFrame(t6)
freq = x6.value_counts()
x7 = pd.DataFrame(freq, columns=["Frequency"])
x8 = x7.reset_index()
x8.rename(columns={0: "Word"}, inplace=True)
x9 = x8.sort_values(by="Word")
x10 = x9.Word.unique()
x10[:100]
```

```
array(['', '-го', '-ой', 'а', 'абсурда', 'августа', 'авто',
       'автоинспекция', 'автомобиль', 'автомобиля', 'авторы', 'агентов',
       'агентство', 'агрегирование', 'агрегирования', 'адрес', 'адреса',
       'аккуратно', 'акра', 'акта', 'активно', 'алфавит', 'алфавите',
       'альтернативное', 'альфа', 'альфа-самец', 'американские',
       'американский', 'американских', 'американской', 'америке',
       'анализ', 'анализа', 'аналитика', 'аналоги', 'анархию',
       'английски', 'английские', 'английского', 'английское',
       'английском', 'анри', 'аня', 'аппаратными', 'арктические',
       'аромат', 'базовых', 'балтики', 'банка', 'банке', 'бар', 'бара',
       'бед', 'без', 'безопасности', 'безработицы', 'бекки', 'берем',
       'берет', 'беру', 'бесплатно', 'бесплатное', 'библиотеки', 'бизнес',
       'бизнес-аналитик', 'бизнес-аналитика', 'бизнес-аналитики',
       'бизнес-процесс', 'бизнес-процессах', 'бизнес-тренер', 'бизнеса',
       'бизнесе', 'билетный', 'бирже', 'блага', 'ближайшее', 'ближе',
       'блинчики', 'блок', 'богатство', 'более', 'болеют', 'больничных',
       'больному', 'больных', 'большая', 'больше', 'большие', 'большим',
       'больших', 'большого', 'большое', 'большой', 'большую', 'бочку',
       'браков', 'брал', 'брокер', 'бросая', 'брускам'], dtype=object)
```

Рис. Очищенный текст

Посмотрим, сколько слов у нас набралось после всех наших манипуляций, см. рис. Мы вызываем свойство **shape**. Так мы можем узнать, какие «размеры» имеет наш объект. В данном примере это 2626 элементов.

```
x10.shape
```

```
(2626,)
```

Рис. Размерность объекта

2.9. ЛЕММАТИЗАЦИЯ

Как мы уже убедились, в тексте встречаются одинаковые слова, но в разных формах. Перед анализом частоты употребления слов их нужно привести к начальной форме. Для существительных это единственное число, именительный падеж. Для прилагательных — мужской род, единственное число, именительный падеж. Такая операция называется стандартизацией, нормализацией или, более конкретно, лемматизацией. ЛЕММА в данном случае — это начальная форма слова.

Создадим новый раздел блокнота и назовём его «Лемматизация».

Мы будем использовать специальную библиотеку **pymorphy**. Для начала установим эту библиотеку, см. рис.

Мы вызываем команду **pip install**, причем не в Питоне, в операционной системе. Утилита **pip** находит и скачивает нужные файлы из интернета. На экран выводятся соответствующие сообщения: **Downloading**...

Задание. Просмотрите в Википедии статью **Pip (менеджер пакетов)**. Выясните, как можно расшифровывать это название.

```
!pip install pymorphy2
!pip install -U pymorphy2-dicts-ru
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels
Collecting pymorphy2
  Downloading pymorphy2-0.9.1-py3-none-any.whl (55 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 55.5/55.5 KB 4.3 MB/s eta 0:00:00
Collecting dawg-python>=0.7.1
  Downloading DAWG_Python-0.7.2-py2.py3-none-any.whl (11 kB)
Collecting pymorphy2-dicts-ru<3.0,>=2.4
  Downloading pymorphy2_dicts_ru-2.4.417127.4579844-py2.py3-none-any.whl (8.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.2/8.2 MB 106.7 MB/s eta 0:00:00
Collecting docopt>=0.6
  Downloading docopt-0.6.2.tar.gz (25 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: docopt
  Building wheel for docopt (setup.py) ... done
  Created wheel for docopt: filename=docopt-0.6.2-py2.py3-none-any.whl size=13721 s
  Stored in directory: /root/.cache/pip/wheels/70/4a/46/1309fc853b8d395e60bafaf1b6d
Successfully built docopt
Installing collected packages: pymorphy2-dicts-ru, docopt, dawg-python, pymorphy2
Successfully installed dawg-python-0.7.2 docopt-0.6.2 pymorphy2-0.9.1 pymorphy2-dic
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels
Requirement already satisfied: pymorphy2-dicts-ru in /usr/local/lib/python3.9/dist-
```

Рис. Установка библиотеки

Теперь можем провести лемматизацию, см. рис.

Создаем экземпляр объекта lemmatizer.

По умолчанию используется словарь русского языка.

Далее создаем пустую строку, к которой будем добавлять шаг за шагом новые леммы.

В цикле перебираем все слова в массиве **t6** и преобразуем их в леммы.

Для работы мы задействуем только один главный элемент результата грамматического разбора **word [0]**. Остальную информацию мы пока не используем.

Полученный текст читать человеку будет тяжеловато. Этот текст предназначен только для машинной обработки, см. рис.

Обратим внимание, что теперь текст звучит немного «неграмотно» — примерно, как «моя твоя не понимай». Здесь все слова приведены к своей исходной форме.

```
import pymorphy2
lemmatizer = pymorphy2.MorphAnalyzer()
t_lemm=''
for word in t6:
  word = lemmatizer.parse(word)
  t_lemm = t_lemm + ' ' + word[0].normal_form
```

```
t_lemm
```

```
' итак название наш предмет бизнес аналитика и статистика это вроде как два разн
е один и тот же профессия один и то же область первый тема мой неделя они как ра
демия быть мы вместо обычный который пока у мы заняться ну к сожаление по просьб
с быть хорошо у мы есть  указать три адрес интернет на сам адрес этот система ди
ряд проходить то чтобы в конец семестр не читать какой-то остаток материал у мы
просто на случай выстаивать временно не быть работать сайт иногда там проводитьс
часть затяжка зум здесь и на начать с конец чем заканчиваться обработк… '
```

Рис. Лемматизация

Посмотрим, что же происходит при лемматизации, см. рис.

Введем всего одно слово: «красными». Проведем грамматический разбор командой **parse**. На жаргоне программистов это называется «распарсить».

Подробнее можно прочитать в руководстве:

Морфологический анализатор pymorphy2. Руководство пользователя:

https://pymorphy2.readthedocs.io/en/stable/user/guide.html[1]

Первый вариант разбора, он же наиболее вероятный: **ADJ** — adjective — прилагательное. Эти сведения находятся в атрибуте **tag** — тег.

Атрибут **normal_form** содержит исходную, нормальную форму слова. В нашем случае «красный».

[1] https://ridero.ru/link/4BR8BIluKTZTMU-ZPviG2

```
morph = pymorphy2.MorphAnalyzer(lang="ru")
kr = morph.parse("красными")
kr
```

```
[Parse(word='красными', tag=OpencorporaTag('ADJF,Qual plur,ablt'), normal_form='красный',
 Parse(word='красными', tag=OpencorporaTag('NOUN,anim,masc plur,ablt'), normal_form='крас
 Parse(word='красными', tag=OpencorporaTag('NOUN,inan,neut,Geox plur,ablt'), normal_form=
```

```
print(kr[0])
print(kr[0].tag)
print(kr[0].normal_form)
```

```
Parse(word='красными', tag=OpencorporaTag('ADJF,Qual plur,ablt'), normal_form='красный',
ADJF,Qual plur,ablt
красный
```

Рис. Внутри лемматизации

У каждого слова в языке много форм. Это и число, и род, и падеж, и многое другое. Подробности в можно найти в словарях, см. рис.

Ссылка на ВИКИ-словарь:

https://ru.wiktionary.org

https://ru.wiktionary.org/wiki/красный

падеж		ед. ч.			мн. ч.
		муж. р.	ср. р.	жен. р.	
Им.		кра́сный	кра́сное	кра́сная	кра́сные
Р.		кра́сного	кра́сного	кра́сной	кра́сных
Д.		кра́сному	кра́сному	кра́сной	кра́сным
В.	одуш.	кра́сного	кра́сное	кра́сную	кра́сных
	неод.	кра́сный			кра́сные
Т.		кра́сным	кра́сным	кра́сной кра́сною	кра́сными
П.		кра́сном	кра́сном	кра́сной	кра́сных
Кратк. форма		кра́сен	кра́сно красно́	красна́	кра́сны красны́

Рис. Формы слова «красный»

Зададим несколько форм слова «красный» и посмотрим, как с ними справится наш инструмент, см. рис.

```
tx = "красный красным красных красное красными"
print (tx)
tz = tx.split(" ")
print(tz)
tt_lemm = ''
for word in tz:
  word = lemmatizer.parse(word)
  tt_lemm = tt_lemm + ' ' + word[0].normal_form
tt_lemm
```

```
красный красным красных красное красными
['красный', 'красным', 'красных', 'красное', 'красными']
' красный красный красный красный красный'
```

Рис. Исследование лемматизации

Задание. Проверьте, как лемматизатор обработает все формы слова «статистика».

2.10. СТОП-СЛОВА

Загружаем библиотеку обработки естественного языка **NLTK** — Natural Language ToolKit.

Нам понадобится список **stop_words**, см. рис.

В буквальном переводе их называют «стоп-словами». Имеются в виду служебные слова, которые нужно удалить перед анализом. То есть слова, которые «нужно остановить на входе», то есть «запрещено пропускать». Сюда относятся предлоги, местоимения, числительные и прочие слова, не имеющие самостоятельного значения для предварительного анализа содержания текста. Можно сказать, что это своеобразная фильтрация.

Задание. Просмотрите в Википедии статьи **Stop word**, **Шумовые слова**.

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words = stopwords.words('russian')
stop_words
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
['и',
 'в',
 'во',
 'не',
 'что',
 'он',
 'на',
 'я',
```

Рис. Загрузка списка «стоп-слов»

Построим облако слов по результатам фильтрации. Как видим, состав облака слов немного изменился, см. рис.

```
# Облако слов с фильтрацией стоп-слов
cloud = WordCloud(width=2000, height=1000,
                  background_color='white',
                  stopwords=stop_words).generate(t_lemm)
plt.figure(figsize=(12,8))
plt.imshow(cloud)
plt.axis('off')
plt.show()
```



Рис. Облако слов с фильтрацией стандартных стоп-слов

Как видим, большинство слов никак не связаны с тематикой анализируемого текста. Можно видеть такие слова как «человек», «самый», «говорить», «видеть» и так далее.

```
# Облако слов
cloud = WordCloud(width=2000, height=1000, background_color='white', stopwords=stop_words).generate(t_lemm)
plt.figure(figsize=(16,8))
plt.imshow(cloud)
plt.axis('off')
```

```
(-0.5, 1999.5, 999.5, -0.5)
```



Рис. Облако слов после фильтрации

Рассмотрим результаты стандартной «фильтрации» стоп-слов, см. рис. Пока что остается много «лишних» слов.

```
t11 = t_lemm.split(" ")
t_filt=''
for word in t11:
  if word not in stop_words:
    t_filt = t_filt + ' ' + word
t_filt
```

```
'  итак название наш предмет бизнес аналитика статистика это вроде раз
ко закончиться спорт эпидемия вместо обычный который пока заняться сож
истема дистанционный изучение готовить  всё помочь всё материал ссылка
лировать сайт орк  просто случай выстаивать временно работать сайт про
я обработка данные представление передавать пример пример выводиться э
й-то обобщённый сведение работа наш пример это какой-то прибыль соотно
```

Рис. Результаты фильтрации

Проверим, входят ли общеупотребительные служебные слова в наш список «стоп-слов». Сформируем вспомогательный массив из нескольких простых слов, таких как «это» и «наш». Проверим вхождение слов в список в цикле и выведем результаты на экран.

Как видим, результаты неутешительные, см. рис.

Такой неполный список вряд ли нас удовлетворит. Тем более, что вряд ли существует такой «универсальный» список слов, ведь для каждой задачи списки будут отличаться. В одной ситуации одни и те же слова могут быть полезными, информативными. А в другой ситуации это будет информационный шум.

```
xx = ["и", "итак", "который", "это", "этот", "наш"]
for word in xx:
  print(f"{word}: {word in stop_words}")
```

```
и: True
итак: False
который: False
это: False
этот: True
наш: False
```

Рис. Проверка списка фильтрации

2.11. ПЕРСОНАЛИЗАЦИЯ СТОП-СЛОВ

Попробуем составить свой список стоп-слов — для нашей конкретной задачи. Для начала нам понадобится список уникальных лемматизированных слов из нашего текста, отсортированный по алфавиту. Разделим строку текста на слова, разделенные пробелами. Чтобы удалить дубликаты, преобразуем список в множество и обратно, см. рис.

```
t_split = t_lemm.split(" ")
print(len(t_split), t_split[:10])
```

```
7206 ['', 'итак', 'название', 'наш', 'предмет', 'бизнес', 'аналитика', 'и', 'статистика', 'это']
```

```
s = set(t_split)
print(len(s), s)
```

```
1749 {'', 'домашний', 'определённый', 'вы', 'с', 'ведение', 'западный', 'поиск', 'нога', 'про', '
```

```
x = list(s)
print(len(x), x[:10])
```

```
1749 ['', 'домашний', 'определённый', 'вы', 'с', 'ведение', 'западный', 'поиск', 'нога', 'про']
```

```
x.sort()
print(len(x), x[:10])
```

```
1749 ['', '-го', '-ой', 'а', 'абсурд', 'август', 'авто', 'автоинспекция', 'автомобиль', 'автор']
```

Рис. Список уникальных словСформируем табличный объект **DataFrame** и запишем его в файл в формате MS Excel. Для этого используем функцию **to_excel**. Указываем название файла, см. рис.

```
pdx = pd.DataFrame(x)
pdx.to_excel("word_list.xlsx")
```

Рис. Записываем список слов в файл

Этот файл располагается в рабочем каталоге в облачном хранилище, но только на время работы данного блокнота. Файл можно найти во вкладке **Files**.

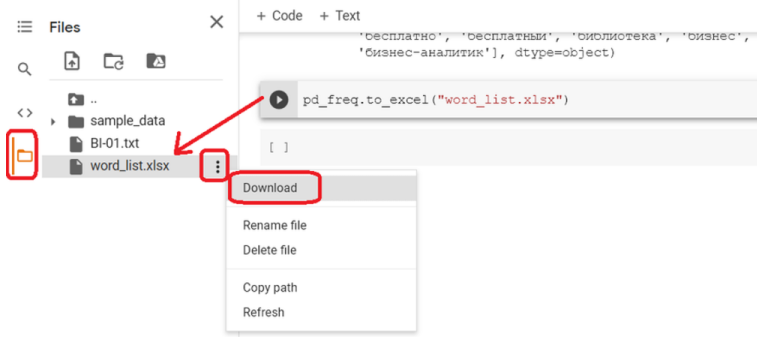


Рис. Вывод в файл

Загружаем файл на локальный компьютер и открываем в табличном редакторе. Удаляем первый столбец с номерами строк, см. рис.

	A	B
1		**0**
2	**0**	
3	**1**	-го
4	**2**	-ой
5	**3**	а
6	**4**	абсурд
7	**5**	август
8	**6**	авто
9	**7**	автоинспекция
10	**8**	автомобиль
11	**9**	автор
12	**10**	агент

	A
1	**0**
2	
3	-го
4	-ой
5	а
6	абсурд
7	август
8	авто
9	автоинспекция
10	автомобиль
11	автор
12	агент

Рис. Список слов в **Excel**

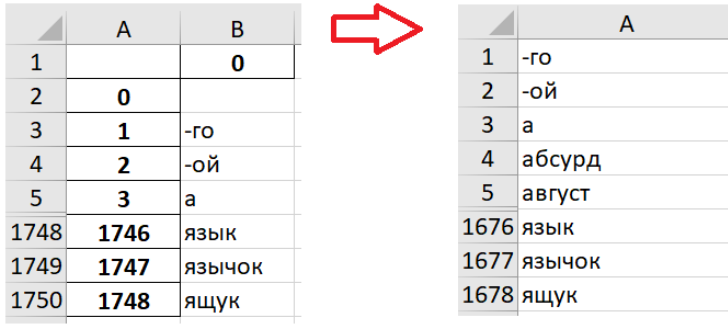
Сохраним таблицу в новом файле и назовем этот файл **stop_words. xlsx**. Оставляем шумовые слова и удаляем из нашей таблицы слова, имеющие отношение к теме документа: «статистика», «экономика», «предприятие» и т. п.

Удаляем ненужную строку целиком. Для этого наводим курсор на заголовок строки, щелкаем правой кнопкой мыши и выбираем **Удалить**.

Конечно, потребуется некоторое время для удаления важных, ключевых слов. В нашем примере их набралось около сотни, см. рис. Этот список нам пригодится и в дальнейшей работе.

Главная проблема — это знания эксперта, способность специалиста выделить важное из большого массива информации.

Кроме того, для каждой темы потребуется свой специализированный набор стоп-слов. Сюда же попадают ошибки написания или распознавания слов, например, «ящук». Итак, список слов стал немного короче, см. рис.



	A	B
1		**0**
2	**0**	
3	**1**	-го
4	**2**	-ой
5	**3**	а
1748	**1746**	язык
1749	**1747**	язычок
1750	**1748**	ящук

	A
1	-го
2	-ой
3	а
4	абсурд
5	август
1676	язык
1677	язычок
1678	ящук

Рис. Сокращённый список

Сохраняем таблицу **Excel** в новом файле **stop_words. xlsx**. Загружаем файл в рабочую область облачного блокнота. Для чтения файла используем функцию **read_excel** из библиотеки **pandas**. Получаем табличный объект **pandas DataFrame**. Просматриваем содержимое с помощью функций **head** и **tail** — несколько первых и последних строк, см. рис.

```
# Читаем файл Excel
pd_stop_words = pd.read_excel("/content/stop_words.xlsx", header=None)
```

```
pd_stop_words.head(5)
```

	0
0	-го
1	-ой
2	а
3	абсурд
4	август

```
pd_stop_words.tail(5)
```

	0
1673	являться
1674	явно
1675	язык
1676	язычок
1677	ящук

Рис. Загрузка файла

Мы указали, что в нашем файле нет строки заголовков для загруженных столбцов: **header=None**. Поэтому в качестве заголовка столбца указан его порядковый номер **0**. Строки тоже начинаются с нулевой. Как мы помним, в Питоне нумерация обычно начинается с нуля.

Извлекаем значения ячеек таблицы с помощью свойства **values**. Преобразуем полученный столбец в объект типа список **tolist**. Выводим на экран начало и конец списка. Здесь мы не используем команду **print**. Просто пишем названия нужных переменных через запятую. Для **Colab** это сработает.

Попутно выясняем размер нашего списка — команда **len** — length — длина (списка).

Сравниваем содержимое таблицы и списка. Убеждаемся, что ничего по дороге не потеряли.

```
my_stop_words = pd_stop_words.values.tolist()
my_stop_words[:5], my_stop_words[-5:], len(my_stop_words)
```

```
([['-го'], ['-ой'], ['а'], ['абсурд'], ['август']],
 [['являться'], ['явно'], ['язык'], ['язычок'], ['ящук']],
 1678)
```

Рис. Преобразуем таблицу в список

Затем в цикле проверяем присутствие каждого слова в нашем специализированном списке стоп-слов my_stop_words.

Просматриваем полученный «отфильтрованный» текст. Теперь здесь остались только информативные ключевые слова, см. рис.

```
"абсурд" in my_stop_words
```

```
False
```

```
["абсурд"] in my_stop_words
```

```
True
```

```
t_filt=''
for word in t11:
  if not ([word] in my_stop_words):
    t_filt = t_filt + ' ' + word
t_filt
```

```
' бизнес аналитика статистика система обработка данные инди
ика статистика статистика обработка статистика статистика биз
ычислять бизнес бизнес программа статистический процесс програ
ый государственный федеральный государственный статистика госуд
тельство правительство гражданин закон статистика закон бизнес
ка международный закон технология стандарт число
затель обобщённый количественный число обобщённый характеристика
...'
```

Рис. Отфильтрованный текст

Нам осталось удалить лишние пробелы в тексте. Для этого построим цикл с предварительным условием **while**. Функция **find** возвращает номер позиции искомой подстроки в исходной строке либо минус единицу, если подстрока не найдена.

Если в тексте есть два пробела подряд, то их нужно заменить на одиночный пробел. Операция поиска и замены replace повторяется до тех пор, пока не закончатся двойные пробелы.

Выводим отфильтрованный текст на экран и видим, что остались только одиночные пробелы, см. рис.

```
while t_filt.find("  ") > -1:
  t_filt = t_filt.replace("  ", " ")
t_filt
```

```
' бизнес аналитика статистика система общий обработка информ
ий объект руководитель управление характеристика продукт про
истика инструмент инструмент статистика бизнес-аналитик стат
изнес бизнес программа инструмент статистический анализ проц
предприятие частный государственный коммерческий бизнес-анал
мерческий деятельность инструмент организация государственны
тельство открытый учреждение правительство закон статистика
...'
```

Рис. Удаление лишних пробелов

Сгенерируем облако слов по отфильтрованному тексту. Можно видеть, что на графике представлены только выбранные ключевые слова, см. рис. Этот вариант был получен вручную.

```
# Облако слов
cloud = WordCloud(width=2000, height=1000, background_color='white').generate(t_filt)
plt.figure(figsize=(12,8))
plt.imshow(cloud)
plt.axis('off')
plt.show()
```



Рис. Отфильтрованное облако слов

Применим маску и перестроим облако слов, см. рис. Сохраняем картинку в файл.

```
# Облако слов по маске
cloud = WordCloud(width=2000, height=1000,
                  background_color='white',
                  mask=circle_mask).generate(t_filt)
plt.figure(figsize=(12,8))
plt.imshow(cloud)
plt.axis('off')
plt.show()
```



Рис. Облако слов по маске

2.12. ГИСТОГРАММА

По существу, облако слов — это наглядное представление гистограммы частот.

Вернемся к исходному тексту и построим для него гистограмму частот. Какие слова встречаются чаще всего? Это простые общеупотребительные «служебные» слова, см. рис.

```
pd_filt = pd.DataFrame(t1.split(" "), columns=["Word"])
Freqs = pd_filt.value_counts()
plt.figure(figsize=(10,5))
Freqs[:30].plot.bar()
plt.xlabel("Слова")
plt.ylabel("Частота слов в тексте")
plt.title("Лекция по бизнес-аналитике и статистике №1")
plt.show()
```

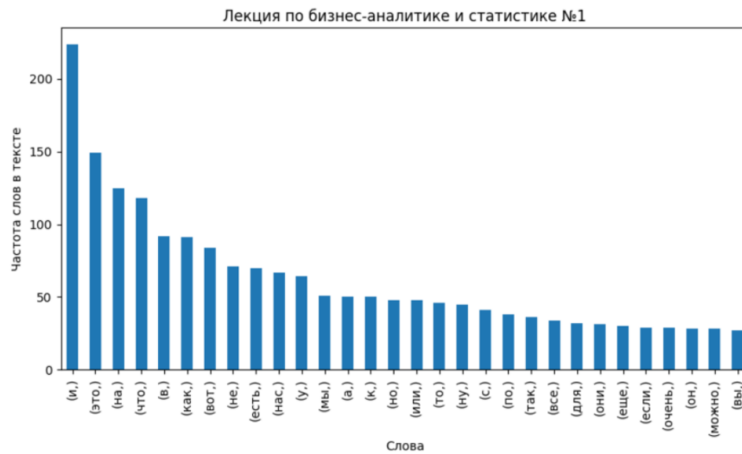


Рис. Гистограмма частот слов исходного текста

Как видим, частота слов довольно быстро убывает. При этом первые 30 слов не несут особого смысла и не подходят в каче-

стве ключевых.

Это явление пытались объяснить и даже использовали для этого слово «закон», хотя здесь далеко до настоящих законов природы.

Задание. Просмотрите в Википедии статью **Закон Ципфа**.

Проведём анализ частоты ключевых слов в нашем отфильтрованным тексте.

Построим столбиковую диаграмму частот, см. рис.

Можно видеть, что первые пять-десять слов встречаются чаще всего. Именно они будут выделены в облаке слов самым крупным шрифтом.

Скорее всего, эти слова соответствуют теме рассматриваемого текста.

```
pd_filt = pd.DataFrame(t_filt.split(" "), columns=["Word"])
Freqs = pd_filt.value_counts()
plt.figure(figsize=(10,5))
Freqs[:30].plot.bar()
plt.xlabel("Ключевые слова")
plt.ylabel("Частота слов в тексте")
plt.title("Лекция по бизнес-аналитике и статистике №1")
plt.show()
```

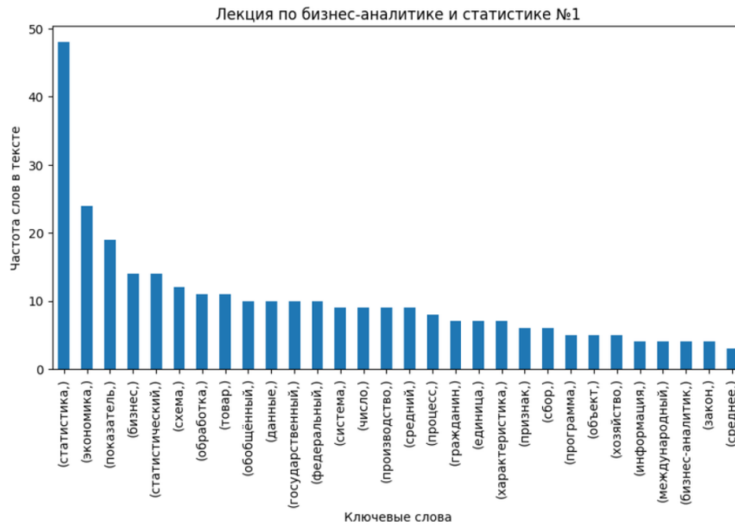


Рис. Гистограмма частот отфильтрованного текста

Задание. Выберите длинное видео на канале автора [1]. Загрузите расшифровку видео и составьте по нему свой список стоп-слов. Постройте облако слов и гистограмму частот. Затем используйте свой список стоп-слов для обработки другого текста на ту же самую тему. Это может быть следующая лекция выбранного цикла. Список стоп-стол можно уточнять и дополнять.

2.13. ЧАСТОТНЫЙ АНАЛИЗ ТЕКСТА КНИГИ

Попробуем повторить основные шаги анализа текста для учебного пособия в формате PDF.

Для опытов используем страницу автора на сайте издательства:

https://ridero.ru/

Переходим в раздел **Магазин**. В строке поиска вводим: **Арьков Валентин**, см. рис.

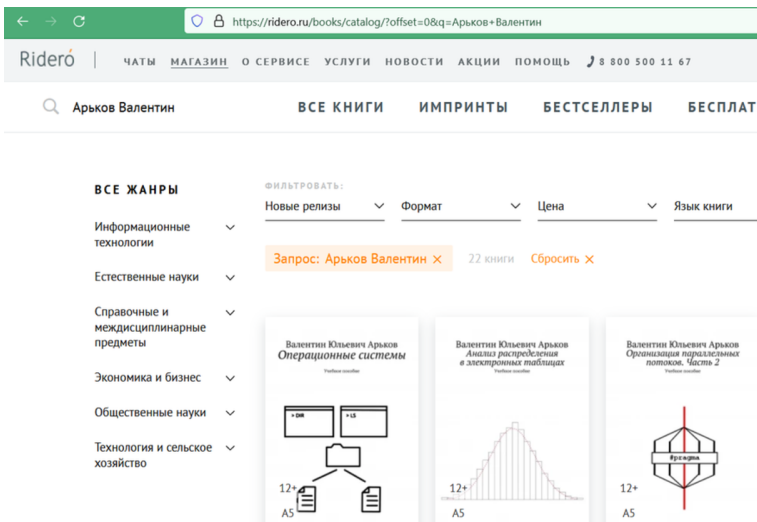


Рис. Страница книг автора

Выбираем первую книгу и бесплатно загружаем вариант

в формате PDF.

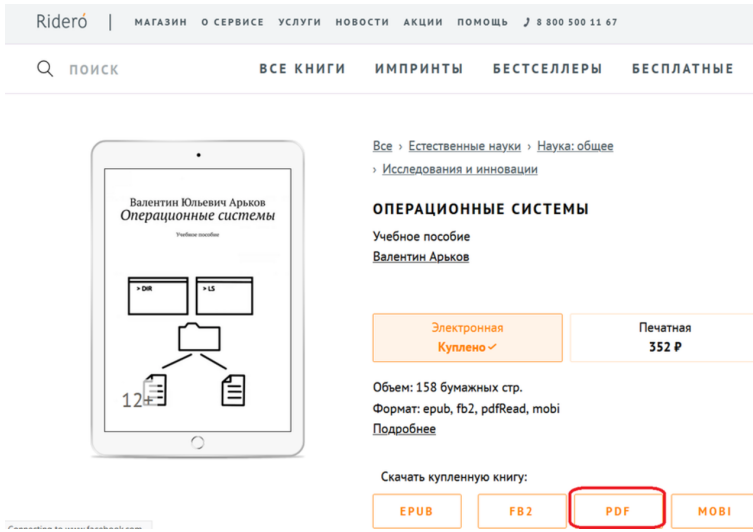


Рис. Загрузка книги

Если открыть файл PDF в Блокноте, мы обнаружим много служебной информации, см. рис. А нам для анализа нужен только текст — без картинок и форматирования.

```
%PDF-1.4
%ïìчÿю
1 0 obj
<< /Outlines 3 0 R /Pages 4 0 R /Type /Catalog >>
endobj
2 0 obj
<< /CreationDate <1b552926cfd72ed0275501b25307b72985> /Creator <2f0b7d6296c62dc
endobj
3 0 obj
<< /Count 66 /First 5 0 R /Last 6 0 R /Type /Outlines >>
endobj
4 0 obj
<< /Count 160 /Kids [ 7 0 R 8 0 R 9 0 R 10 0 R 11 0 R 12 0 R 13 0 R 14 0 R 15 0 R 16 0 R 1
7 0 R 48 0 R 49 0 R 50 0 R 51 0 R 52 0 R 53 0 R 54 0 R 55 0 R 56 0 R 57 0 R 58 0 R 59 0 R 6
90 0 R 91 0 R 92 0 R 93 0 R 94 0 R 95 0 R 96 0 R 97 0 R 98 0 R 99 0 R 100 0 R 101 0 R 102
R 129 0 R 130 0 R 131 0 R 132 0 R 133 0 R 134 0 R 135 0 R 136 0 R 137 0 R 138 0 R 139 0
```

Рис. Содержимое файла PDF

Откроем PDF с помощью текстового редактора MS Word. Последние версии позволяют открыть такой формат на редактирование. Затем сохраняем его как обычный текст в формате *.TXT. На всякий случай откроем файл в Блокноте и сохраним в формате UTF-8.

Затем загружаем фал по проверенной технологии, см. рис.

```
ff = open('/content/OS.txt', "r")
t0 = ff.read()
```

```
t0[:100]
```

```
'\n    \nВалентинЮльевичАрьков\nОперационныесистемы\nУчебноепособие\nИздательские решения\nПо лицензии Rider'
```

```
t0 = t0.replace('\n',' ')
t0[:100]
```

```
'     ВалентинЮльевичАрьков Операционныесистемы Учебноепособие Издательские решения По лицензии Rider'
```

+ Code + Text

```
t0 = t0.replace('  ',' ')
t0[:100]
```

```
' ВалентинЮльевичАрьков Операционныесистемы Учебноепособие Издательские решения По лицензии Ridero 20'
```

Рис. Загружаем файл

Построим облако слов. Для начала — без фильтрации, см. рис.

```
wordcloud = WordCloud(width=2000, height=1000, background_color='white').generate(t0)
plt.figure(figsize=(10,6))
plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```



Рис. Облако слов без фильтрации

Дальнейшие действия понятны. Мы их уже рассмотрели.

Задание. Загрузите учебное пособие автора на сайте **Ридеро** [2]. Постройте облако слов и гистограмму частот.

БЛАГОДАРНОСТЬ

Автор выражает признательность организаторам программы переподготовки МИФИ и Университета 2030 за оригинальные идеи и демонстрацию технологий, которые были использованы в данной работе.

ССЫЛКИ

1. Канал автора на Youtube для анализа видео:
https://www.youtube.com/@ACYTV
2. Книги автора на сайте Ридеро:
https://ridero.ru/books/catalog/?offset=0&q=Арьков+Валентин[1]
3. Пример авторской программы для анализа распределения на Python:
https://github.com/Valentin-Arkov/Statistics/blob/master/Frequency_Distribution_Analysis.ipynb[2]

[1] https://ridero.ru/link/sLtULjOhYJaPtKFT_s__G
[2] https://ridero.ru/link/8oof6KHRqg7GXzXDy-Jbs

ЛИТЕРАТУРА

Марк Лутц. Python. Карманный справочник. М.: Диалектика, 2019. 320 с.

Марк Лутц. Изучаем Python. Том 1. М.: Диалектика, 2019. 832 с.

Марк Лутц. Изучаем Python. Том 2. М.: Вильямс, 2020. 720 с.

Майкл Доусон. Программируем на Python. СПб.: Питер, 2020. 416 с.

Эрик Мэтиз. Изучаем Python. СПб.: Питер, 2021. 512 с.

Дж. Вандер Плас. Python для сложных задач. Наука о данных и машинное обучение. СПб.: Питер, 2018. 576 с.

Уэс Маккинни. Python и анализ данных. М.: ДМК Пресс, 2015. 484 с.

Джоэл Грас. Data Science. Наука о данных с нуля. СПб.: БХВ-Петербург, 2020. 411 с.

ПОСЛЕСЛОВИЕ

Если читателю удалось добраться до конца текста и при этом освоить материал — поздравляем!

Можно похвалить себя и отметить данное событие.

Можно даже материально поддержать наш проект по изданию учебных материалов, покупая наше пособие по цене читателя на сайте издательства **Ридеро**. Но это только по желанию.

До новых встреч!

Валентин Юльевич Арьков

Частотный анализ числовых и текстовых данных

Учебное пособие

Создано в интеллектуальной издательской системе Ridero

Аффилиация: Арьков Валентин Юльевич — доктор технических наук, профессор кафедры автоматизированных систем управления Уфимского университета науки и технологий. Опыт преподавания в вузе более 30 лет.

Данная работа посвящена применению современных методов и технологий больших данных в курсе бизнес-аналитике и статистики. Анализ частоты появления различных значений — один из способов первоначальной обработки данных. Чаще всего его относят к описательной статистике или аналитике. Подсчет частот имеет отношение распределению. Поэтому для успешного восприятия материала желательно освежить в памяти основы теории вероятностей и математической статистики.

ISBN 978-5-0059-8841-6



Rideró

Rideró.ru — издай книгу бесплатно!